

A COMPANION FOR THE NON-SPECIALIST

How This Robot Knows Where It Is

Probability, Particle Filters, AMCL, Nav2
and every number in the evaluation, worked by hand

Promise Emeziem Adiole

Multimodal Robot Autonomy — ROMR in Gazebo Classic

Cyber-Physical Systems Group, Montanuniversität Leoben

Université Marie et Louis Pasteur — ARMAC

How to read this

This document exists for one reason: so that every number in the evaluation of this project can be understood, checked and repeated by somebody who has never met a particle filter. It assumes secondary-school mathematics — you should be comfortable with a square root, a fraction and a graph — and nothing else. Every symbol is defined the first time it appears, and every integral is explained in words before it is written down.

The document is organised so you can enter it at three depths.

Three ways through

The fast path (about 40 minutes). Read Part 1 §1.1–1.3, Part 2 §2.1–2.2, Part 7 in full, and Part 8. That is enough to understand what was measured and to reproduce the headline results with a calculator.

The full path. Read it front to back. Part 1 builds probability from nothing; Parts 2–5 build the localisation system on top of it; Parts 6–8 connect that to the software and the measurements; Parts 9–11 tell you where the files are, how to run everything, and what is still worth tuning.

The reference path. Part 9 is a file map. Part 10 is a runbook. Part 12 is a glossary. Use the table of contents.

Three conventions are used throughout.

- Boxes with a **teal** edge state an idea you should carry forward.
- Boxes with an **amber** edge contain arithmetic you can do yourself on paper. Every one of them uses real numbers from this project’s result files, not invented examples.
- Boxes with a **rust** edge flag a place where the obvious conclusion is wrong, or where a claim needs a caveat.

One honesty note, stated once and meant throughout

Every result in this project was obtained in *simulation*. The ROMR platform exists physically — it was built by the project supervisor, from a CAD model produced by another intern — but all navigation, all localisation and all of the numbers below come from a Gazebo Classic model of that robot. Nothing here should be read as a measurement on hardware.

Contents

How to read this	1
1 Probability, from nothing	5
1.1 What a probability actually is	5
1.2 Random variables, discrete and continuous	5
1.2.1 Discrete: countable outcomes	6
1.2.2 Continuous: a value on a ruler	6
1.3 Mean, median, variance and σ	6
1.3.1 Mean (average)	6
1.3.2 Median	7
1.3.3 Variance and standard deviation	7
1.4 The Gaussian	8
1.5 Conditional probability and Bayes' rule	8
1.6 Independence, and why it lets us multiply	10
2 The robot's problem: where am I?	11
2.1 Two different jobs: building the map, and using it	11
2.1.1 Phase 1: mapping, done once	11
2.1.2 Phase 2: operation, every single trial in this study	11
2.1.3 Filtering and smoothing — the difference that explains the results	11
2.2 Three frames, and the seam between them	12
2.3 Odometry: counting wheel turns	14
2.3.1 Why rotation is the enemy on this robot	15
2.4 The belief, and the Bayes filter	15
2.4.1 Step 1 — Prediction (the robot moved)	15
2.4.2 Step 2 — Correction (the laser saw something)	16
2.4.3 Why we cannot just compute this	16
3 Monte Carlo: a belief made of dots	17
3.1 The idea: replace a curve with a crowd	17
3.2 Importance sampling: the mathematics that makes it legal	17
3.2.1 The problem	17
3.2.2 The trick	18
3.2.3 The precondition nobody mentions, and why it matters here	18
3.3 The three steps of a particle filter	18
3.3.1 Why Step 3 is necessary	19
3.4 Seeing it happen on this robot	19
4 Resampling: the wheel, and the better wheel	21
4.1 The cumulative distribution: turning weights into a ruler	21
4.2 Method A — the roulette wheel (multinomial resampling)	21
4.2.1 How much does it miss by? (exact answer)	22
4.3 Method B — stochastic universal sampling (systematic, low variance)	22
4.3.1 The guarantee, proved	23
4.3.2 The variance, quantified	24
4.3.3 Running cost	24
4.4 Which one does this project actually use, and why?	25

4.4.1	What KLD adaptive sampling is, and why it blocks Method B	25
4.4.2	The actual incompatibility	26
4.4.3	What this means for this project's results	27
5	AMCL: every knob, and what it does	28
5.1	The split: what you choose, and what chooses you	28
5.2	The proposal: the odometry motion model and the five alphas	28
5.3	The tuning that went backwards, and what it teaches	29
5.4	The weight: the measurement model	31
5.4.1	The other two weight faults	31
5.5	Divergence recovery, and why it is off here	32
5.6	The remaining parameters	33
5.7	What AMCL reports back, and the covariance	33
6	Nav2: the stack around the filter	34
6.1	What Nav2 is	34
6.1.1	Server by server	34
6.1.2	Which planning algorithm — and why not RRT	35
6.1.3	Three places this system samples, for three different reasons	36
6.2	Where AMCL is in that picture — and why it matters	36
6.3	Costmaps, footprint and inflation	36
6.3.1	Footprint, not radius	37
6.4	The goal checker and the progress checker	38
6.5	What sits on top: the language layer	38
7	How the evaluation actually works	41
7.1	The one decision the whole evaluation rests on	41
7.2	Three distances, and the triangle between them	41
7.3	Arrival: what counts as having got there	42
7.4	The command suite	42
7.5	Trial independence, and why re-seeding is itself a result	43
7.6	What the RViz window shows	43
8	Every number, worked by hand	45
8.1	The distance formula, twice, on real coordinates	45
8.2	Checking the recorded data against itself	46
8.3	Table — validity audit: 16 trials, not 54	47
8.4	Table — reported versus actual arrival, initial run	48
8.5	Table — arrivals as a function of the threshold	49
8.6	Table — before and after correction	49
8.7	The pattern nobody had looked for: error against distance	51
8.7.1	Putting a number on “almost the same list”	51
8.7.2	Why this matters more than the coefficient does	52
8.7.3	And after the corrections	53
8.8	Table — the confusion matrix, precision and recall	53
8.9	Table — the ten unconfirmed arrivals	56
8.10	Numbers that cannot be done by hand — and how they are obtained	56
8.10.1	Scan-map agreement	57
8.10.2	Localisation step response and the α sweep	57
8.10.3	Goal reachability and costmap occupancy	58

9	Where everything lives	59
9.1	Result files	59
9.2	Configuration	60
9.3	Scripts	60
9.4	Backend code that matters to this document	61
10	Running it: from a cold machine to a results CSV	62
10.1	Two settings that are not optional on this host	62
10.2	Start from a clean slate — always	62
10.3	The five terminals	63
10.4	Localise the robot	64
10.5	Watch what is happening	64
10.6	Run the evaluation	64
10.7	Produce the figures	65
10.8	Adding or moving a destination	65
11	What was tuned, and what is left	66
11.1	The four faults that were fixed	66
11.2	Tuning levers not yet exhausted	66
11.2.1	Lever 1 — the progress checker (highest expected value)	67
11.2.2	Lever 2 — resampling interval	67
11.2.3	Lever 3 — update thresholds during rotation	67
11.2.4	Lever 4 — sharpening the likelihood	68
11.2.5	Lever 5 — using the covariance as a gate	68
11.2.6	Lever 6 — inflation, currently uncommitted	68
11.2.7	Lever 7 — the resampler itself	69
11.3	A protocol for tuning anything on this list	69
12	Glossary	70
	Sources for the reproduced material	72

Probability, from nothing

Everything in this project rests on one idea: the robot does not know where it is, so instead of storing a position it stores *how strongly it believes each possible position*. To make that precise we need probability. This part builds it from the beginning.

1.1 What a probability actually is

A **probability** is a number between 0 and 1 that says how strongly we expect something. Zero means impossible. One means certain. Nought point five means “as likely as not”.

The most useful way to think about it for our purposes is the **frequency interpretation**: if you could repeat a situation a very large number of times, the probability of an outcome is the fraction of those repetitions in which the outcome happens.

Probability as a counted fraction — with our own data

In the corrected evaluation run the navigation stack reported “succeeded” on 50 of the 61 commands it was given. If you picked one of those 61 trials at random, the probability that it was a reported success is

$$P(\text{reported success}) = \frac{\text{number of reported successes}}{\text{total trials}} = \frac{50}{61} = 0.8197\dots \approx 82\%.$$

That is all a probability is: a count divided by a total. Nothing more mysterious is happening anywhere in this document.

Three rules follow, and they are the only rules of probability there are. Write A for some event (say, “the robot is in the kitchen”).

1. **Non-negativity.** $P(A) \geq 0$. You cannot believe something a negative amount.
2. **Normalisation.** $P(\text{something happens}) = 1$. The robot is *somewhere*.
3. **Additivity.** If A and B cannot both happen, then $P(A \text{ or } B) = P(A) + P(B)$. The robot cannot be in the kitchen and the garage at once, so the chance it is in one of them is the sum.

Key idea

Rule 2 is the one that will do the most work later. Every time we adjust the robot’s beliefs, we will have to divide by something to force the total back to 1. That division has a name — **normalisation** — and when you see a fraction with a sum in the denominator, that is what it is doing.

1.2 Random variables, discrete and continuous

A **random variable** is just a quantity whose value we are uncertain about. We write it with a capital letter, X , and a particular value it might take with a small letter, x .

There are two kinds, and the distinction matters enormously for robots.

1.2.1 Discrete: countable outcomes

If X can only take separately countable values — which of eight rooms the robot was sent to, whether a trial succeeded or not — it is **discrete**. We describe it by listing a probability for each value:

$$P(X = x_i) = p_i, \quad p_i \geq 0, \quad \sum_i p_i = 1.$$

The capital sigma $\sum$ means “add up all of these”. The condition $\sum_i p_i = 1$ is Rule 2 written in symbols.

1.2.2 Continuous: a value on a ruler

If X can take any value on a continuous scale — the robot’s x coordinate in metres, its heading in radians — it is **continuous**, and something strange happens. The probability that the robot is at *exactly* $x = 2.7183$ m, to infinite decimal places, is zero. There are infinitely many positions and each individual one has no width.

So we do not ask for the probability of a point. We ask for the probability of an *interval*, and we get it from a **probability density function**, written $p(x)$.

The definition you need

$$P(a \leq X \leq b) = \int_a^b p(x) dx.$$

Read this out loud as: *the probability that X lands between a and b equals the area under the curve p between a and b .*

The symbol $\int_a^b \dots dx$ is an **integral**. It means “add up infinitely many infinitely thin slices, from a to b ”. Each slice is a rectangle of height $p(x)$ and width dx ; the integral is their total area. If you have never met an integral before, the only thing you need to believe is that **an integral is an area**, and that an area is a total.

Rule 2 — “the robot is somewhere” — now reads

$$\int_{-\infty}^{\infty} p(x) dx = 1. \quad (1.1)$$

The total area under a density is always exactly one. This is the single most important equation in this document, because almost every difficulty in localisation is a difficulty in keeping equation (1.1) true.

A density is not a probability

$p(x)$ can be bigger than 1. If the robot’s position is known to within a millimetre, the density spikes to a huge value over a tiny width, and the *area* is still 1. Density has units of “per metre”; probability has no units. Confusing the two is the commonest beginner error.

1.3 Mean, median, variance and σ

Four summary numbers appear constantly in the results tables. Here is exactly what each one is.

1.3.1 Mean (average)

Add the values, divide by how many there are:

$$\bar{v} = \frac{1}{n} \sum_{i=1}^n v_i.$$

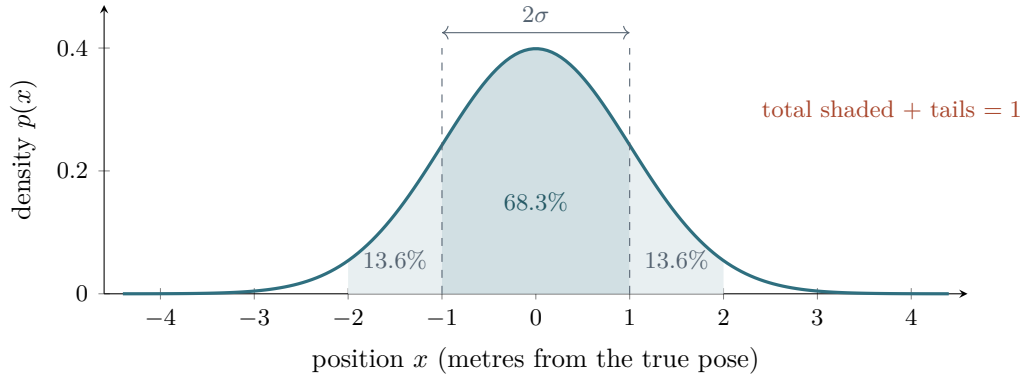


Figure 1.1: A probability density over the robot’s position error. The curve itself is not a probability; the *area* under a stretch of it is. The whole area is 1, because the robot is somewhere. The width parameter σ is what AMCL reports back to us as its own confidence, and what appears in this project’s logs as `sigma_x`, `sigma_y` and `sigma_yaw`.

For a continuous quantity the sum becomes an integral, and the mean is called the **expected value**:

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x p(x) dx. \quad (1.2)$$

Read equation (1.2) as: *go along the axis; at each position x , multiply that position by how much belief sits there; add it all up.* It is a weighted average where the weights are the density. This one equation is the reason particle filters exist, and we will come back to it in Part 3.

1.3.2 Median

Sort the values from smallest to largest and take the middle one. If there is an even number of values, average the two in the middle.

Why the results tables report medians, not means

Consider the sixteen executed trials of the *initial* run. Their true position errors, in metres, sorted:

0.330, 0.426, 0.467, 1.693, 2.071, 5.127, 5.182, **5.202**,
7.491, 7.524, 8.733, 8.793, 11.264, 15.612, 15.631, 15.664.

There are 16 values, so the median is the average of the 8th and 9th:

$$\text{median} = \frac{5.202 + 7.491}{2} = \frac{12.693}{2} = 6.3465 \text{ m} \approx 6.35 \text{ m}.$$

That is the 6.35 m printed in the results table. Now compute the mean of the same sixteen: it is 6.95 m. The mean is dragged upwards by the three 15 m failures. The median tells you what a *typical* trial looked like; the mean tells you about the worst ones. Both are reported, deliberately.

1.3.3 Variance and standard deviation

Variance measures spread. Take each value’s distance from the mean, square it (so that being below and being above both count as spread), and average those squares:

$$\text{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2] = \int_{-\infty}^{\infty} (x - \mu)^2 p(x) dx, \quad \mu = \mathbb{E}[X].$$

Variance is in squared units — metres squared — which is awkward, so we take its square root and get the **standard deviation**:

$$\sigma = \sqrt{\text{Var}[X]}.$$

σ is back in metres (or radians), and it is directly readable: it is roughly “how far off you should typically expect to be”.

What $\sigma_\theta = 1.189$ rad means

In this project’s step-response measurement, one 90° turn drove the heading standard deviation to 1.189 rad. One radian is about 57.3° , so $1.189 \text{ rad} \approx 68^\circ$. The filter was saying: *my best guess at which way the robot is pointing could easily be 68° wrong*. A robot that does not know which way it is facing to within 68° cannot navigate. That single number is the diagnosis of the whole initial failure.

1.4 The Gaussian

One density appears more than any other, because sums of many small independent errors tend towards it. It is the **Gaussian** or **normal** distribution, drawn in Figure 1.1:

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right). \quad (1.3)$$

Term by term:

- μ (mu) is the **centre** — where the peak sits.
- σ (sigma) is the **width** — large σ means a broad, low, uncertain hump; small σ means a narrow, tall, confident spike.
- $(x - \mu)^2$ measures how far x is from the centre; dividing by $2\sigma^2$ turns that into “how many widths away, squared”.
- $\exp(-\dots)$ takes that and turns it into a number that falls off fast — being two widths away is much less than half as likely as being one.
- $1/\sqrt{2\pi\sigma^2}$ is the **normalising constant**. It is exactly the number needed to make the total area equal 1, satisfying equation (1.1). Nothing about it is deep; it is bookkeeping.

The useful rule of thumb, visible in Figure 1.1: about 68% of the belief lies within $\pm 1\sigma$ of the centre, about 95% within $\pm 2\sigma$, and about 99.7% within $\pm 3\sigma$. The number 3 will reappear in Part 4 as AMCL’s `pop_z` parameter, and that is why.

1.5 Conditional probability and Bayes’ rule

This is the engine of the entire project.

Conditional probability, written $P(A | B)$ and read “the probability of A *given* B ”, is the probability of A once you know that B has happened. Its definition is

$$P(A | B) = \frac{P(A \text{ and } B)}{P(B)}.$$

Rearranging that definition twice — once for $P(A \text{ and } B)$ and once for $P(B \text{ and } A)$, which are the same thing — gives **Bayes’ rule**:

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)} \quad (1.4)$$

The four pieces have names, and you should learn them because they are used without explanation everywhere in robotics:

Piece	Name	In our robot
$P(A)$	prior	what we believed about position <i>before</i> the laser scan
$P(B A)$	likelihood	how well the scan matches the map, if we were at A
$P(A B)$	posterior	what we believe about position <i>after</i> the scan
$P(B)$	evidence	the normalising divisor that makes it all sum to 1

Bayes' rule on a school-level example, then on the robot

The school example. A test for a condition is 99% accurate. The condition affects 1 person in 1000. You test positive. What is the chance you have it?

Let A = “you have it”, B = “you test positive”. $P(A) = 0.001$. $P(B | A) = 0.99$. For $P(B)$, add the two ways to test positive:

$$P(B) = \underbrace{0.99 \times 0.001}_{\text{true positive}} + \underbrace{0.01 \times 0.999}_{\text{false positive}} = 0.00099 + 0.00999 = 0.01098.$$

Then

$$P(A | B) = \frac{0.99 \times 0.001}{0.01098} = \frac{0.00099}{0.01098} = 0.0902 \approx 9\%.$$

Only 9%, because the condition is rare and there are far more people available to be false positives. **This is exactly the situation the robot was in.**

The robot version, using our own initial run. Let A = “the robot actually arrived (within 0.5 m)”, B = “the stack reported success”. Over the 16 executed trials of the initial run: 3 arrived, and of those 3, two were reported. Twelve were reported in total.

$$P(A) = \frac{3}{16} = 0.1875, \quad P(B | A) = \frac{2}{3} = 0.667, \quad P(B) = \frac{12}{16} = 0.75.$$

$$P(A | B) = \frac{0.667 \times 0.1875}{0.75} = \frac{0.125}{0.75} = 0.1667 \approx 17\%.$$

So in the initial system, hearing “succeeded” gave you a 17% chance the robot was really there. That number, 0.167, is the **precision** in the results table — and Bayes' rule is where it comes from. Part 8 recomputes it directly from counts and gets the same answer.

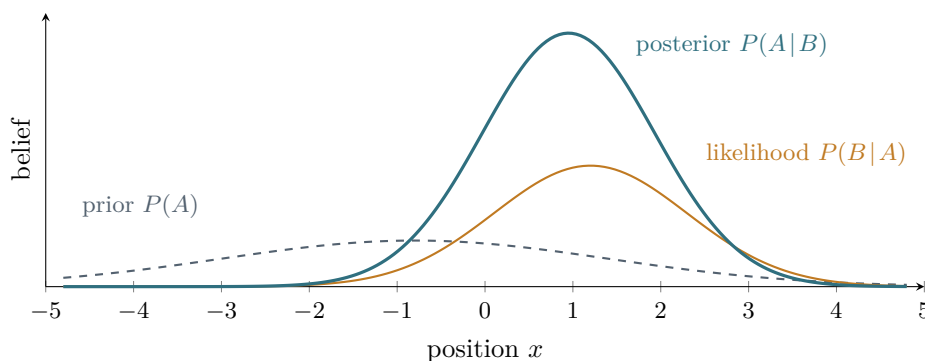


Figure 1.2: Bayes' rule, drawn. The broad dashed curve is what we believed before the measurement. The amber curve is what the laser scan says. Multiplying them point by point — and then rescaling so the area is 1 again — gives the taller, narrower teal curve. Every measurement makes the belief sharper. Every movement, as we will see, makes it broader again.

1.6 Independence, and why it lets us multiply

Two events are **independent** if knowing one tells you nothing about the other. For independent events,

$$P(A \text{ and } B) = P(A) \times P(B).$$

This matters because a laser scan is not one measurement but 120 of them (this project’s `max_beams` setting). AMCL treats the beams as independent given the pose, so the likelihood of a whole scan $\mathbf{z} = \{z_1, \dots, z_{120}\}$ is the product of the individual beam likelihoods:

$$p(\mathbf{z} \mid \mathbf{x}, m) = \prod_{k=1}^{120} p(z_k \mid \mathbf{x}, m), \quad (1.5)$$

where $\prod$ means “multiply all of these together”, $\mathbf{x}$ is the pose being tested, and m is the map.

Independence here is a convenient fiction

Neighbouring laser beams hitting the same flat wall are obviously *not* independent — if one is long, its neighbour almost certainly is too. Multiplying them anyway makes the resulting likelihood far too confident, too peaked. This is a known and accepted approximation in every implementation of this algorithm, and it is one reason a filter can lock onto a wrong hypothesis with great conviction. It is also part of why this project caps the scan at 120 beams out of the 720 available rather than using all of them: fewer, more widely spaced beams are closer to independent.

The robot's problem: where am I?

2.1 Two different jobs: building the map, and using it

Before anything else, a distinction that causes more confusion than any other in this subject, because the two things use the same sensors, produce the same kind of answer, and are almost always drawn in the same diagram.

The question is whether you already have a map

SLAM — Simultaneous Localisation And Mapping — is the problem you have when there is *no* map. You must work out where the robot is *and* draw the building, at the same time, from the same data. Each answer depends on the other, which is what makes it hard.

Localisation is the problem you have when the map already exists. The building is given. All you have to find is the robot inside it.

This project does both — but never at the same time, and never in the same run.

2.1.1 Phase 1: mapping, done once

The robot was driven around the house by hand while `slam_toolbox` was running. At that moment no map existed. `slam_toolbox` produced both the map and the trajectory, and the finished occupancy grid was saved to disk as `custom_map_v2.pgm`. Then that phase ended, permanently. It is not repeated for experiments.

2.1.2 Phase 2: operation, every single trial in this study

`slam_toolbox` is not running. A node called `map_server` loads the saved `.pgm` file, and `amcl` finds the robot inside it. Every number in Part 8 of this document comes from this phase. Nothing in the results measures SLAM.

They cannot both run

Only one node in a ROS system is allowed to publish the map $\rightarrow$ odom transform (§2.2 explains what that is). Both `slam_toolbox` and `amcl` want to publish it. So you run one or the other. There is no configuration in which they cooperate.

2.1.3 Filtering and smoothing — the difference that explains the results

There is a second distinction hiding underneath the first, and it is the one that actually explains why this project's errors look the way they do.

Filtering	Smoothing
Estimates the state <i>now</i> , from the latest measurement.	Estimates the <i>whole trajectory</i> , using every measurement ever taken.
Keeps one answer and moves it forward.	Keeps all the answers and adjusts them together.
Cannot revisit a past mistake — the past is not stored.	Can go back and correct the past when new evidence arrives.
Cheap, constant work per step.	Expensive, and grows as the trajectory grows.
Examples: Kalman filter, extended Kalman filter, particle filter .	Examples: pose-graph optimisation , bundle adjustment.
This project: AMCL, every trial.	This project: slam_toolbox, when the map was built.

How smoothing actually works. A pose graph is a picture of the robot's history. Each **node** is a pose the robot occupied. Each **edge** is a measured relationship between two poses — “from here, I moved forward 0.8 m and turned 10°”. Odometry supplies edges between consecutive poses, and those edges accumulate error, so the drawn trajectory slowly bends away from the truth.

Then comes the event that makes the whole thing work. The robot returns somewhere it has already been, and the laser recognises it. That is a **loop closure**, and it adds an edge between two nodes that are far apart in time. Now the graph is over-determined: it contains a cycle whose edges do not quite agree. **Pose-graph optimisation** — solved here by the Ceres library — moves *every* node a little so that the disagreement is spread evenly around the loop instead of piling up at one end. Poses recorded minutes ago are silently corrected by something observed just now.

Why this matters for Part 8

A smoother can be wrong and then stop being wrong. A filter can only be wrong and stay wrong.

AMCL is a filter. It holds a belief about where the robot is *right now* and nothing else. Once its particles have crowded onto the wrong hypothesis, there is no stored past to re-examine and no mechanism to reconsider — and the crowding itself has thrown away the particles that were near the right answer. This is the whole reason the failures in this study are not merely large but *stable*: in §8.7 you will see three separate runs to the same room finish 15.6 m from the goal and agree with each other to 0.05 m. The filter was not lost and wandering. It was confidently, consistently, repeatably in the wrong place.

§5.5 covers the one escape hatch AMCL provides for this, and why it is switched off here.

2.2 Three frames, and the seam between them

A robot cannot talk about “position” without saying *position measured from what*. A **frame** is an origin plus a set of axes. This project uses three, and understanding the relationship between them explains almost every failure that was diagnosed.

The three frames

base_link	Attached to the robot itself. Its origin sits between the drive wheels; x points forward. When the robot moves, this frame moves with it. Everything mounted on the robot — the laser, the camera — is described relative to base_link .
odom	A fixed frame the robot invents when it switches on. It is <i>smooth</i> : the robot's

position in `odom` never jumps, it only ever changes a little bit at a time. But it *drifts*: after driving for a while, `odom` no longer lines up with the real world.

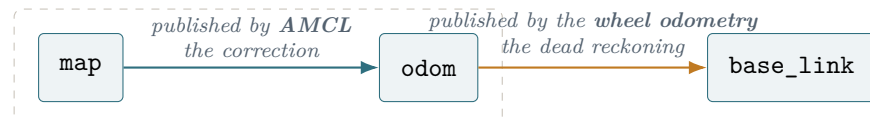
`map`

The world frame, tied to the building. It does *not* drift. But the robot's position in `map` can jump, because it is inferred rather than integrated.

These are chained together. In ROS the chain is

`map` $\rightarrow$ `odom` $\rightarrow$ `base_link`.

this is the only thing AMCL ever produces



fixed to the building
never drifts, can jump

invented at start-up
never jumps, always drifts

rides on the robot

Figure 2.1: The transform chain. The wheel odometry supplies `odom`→`base_link` by counting wheel rotations. AMCL supplies `map`→`odom`, and *that transform is its entire output*. AMCL never moves the robot and never tells anyone where the robot is directly; it silently adjusts the offset between the drifting frame and the true one. Nav2 reads the composed chain and never has to know AMCL exists. Part 6 shows why that seam is where the project's headline fault was hiding.

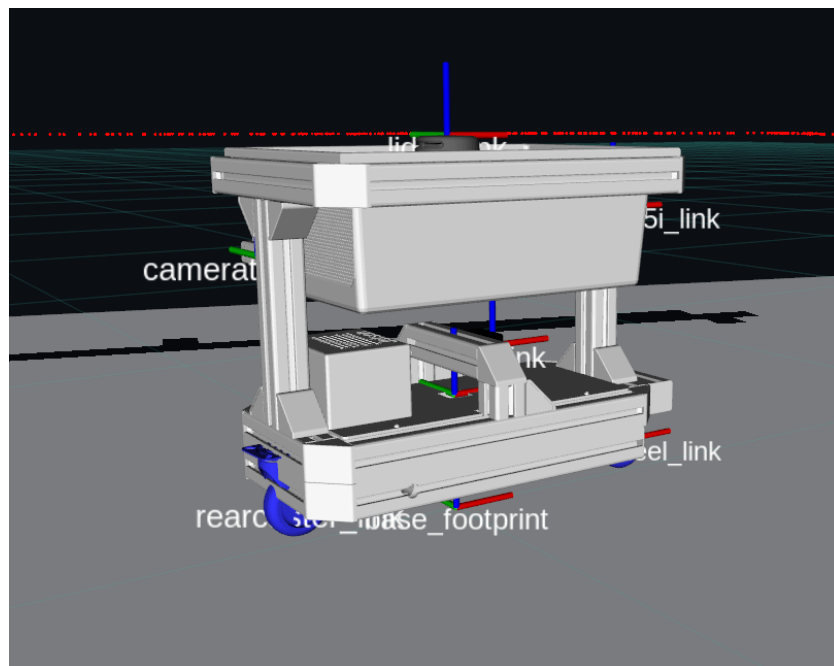


Figure 2.2: The ROMR model in RViz with its coordinate frames drawn. Each little red-green-blue triad is one frame: red is x , green is y , blue is z . `base_footprint` sits on the ground between the wheels, `base_link` on the chassis above it, and the sensor frames (`camera_link`, the lidar frame) are fixed offsets from it. These relationships come from the URDF and never change. The two that *do* change — `map`→`odom` and `odom`→`base_link` — are the ones in Figure 2.1, and they are the entire subject of this document.

Key idea

AMCL's job is not “find the robot”. Its job is “compute the correction that must be applied to the drifting odometry frame so that the composed result matches the map”. Everything else in the navigation stack reads the composed result and takes it on trust.

2.3 Odometry: counting wheel turns

ROMR is a **differential-drive** robot: two independently driven wheels on a common axle, plus passive casters for balance. From the Gazebo plugin configuration used in this project (`romr_gazebo_plugins.gazebo`):

Quantity	Symbol	Value in this project
Wheel separation (track width)	L	0.2937 m
Wheel diameter	$2r$	0.1627 m
Wheel radius	r	0.081 35 m
Odometry publication rate	—	50 Hz

If the left wheel turns at angular rate ω_L and the right at ω_R (radians per second), their contact points move at $v_L = r \omega_L$ and $v_R = r \omega_R$. The robot's body then has

$$v = \frac{v_R + v_L}{2}, \quad \omega = \frac{v_R - v_L}{L}, \quad (2.1)$$

where v is forward speed in m/s and ω is turning rate in rad/s.

Read equation (2.1) in words. *Forward speed is the average of the two wheels* — if both go forward at the same speed, the robot goes forward and does not turn. *Turning rate is the difference divided by the track width* — if the right wheel outruns the left, the robot swings left, and the narrower the robot, the faster it swings for the same difference.

To get position we **integrate**: we add up all the little movements. Over a short interval Δt ,

$$\begin{aligned} \theta_{t+1} &= \theta_t + \omega \Delta t, \\ x_{t+1} &= x_t + v \Delta t \cos \theta_t, \\ y_{t+1} &= y_t + v \Delta t \sin \theta_t. \end{aligned} \quad (2.2)$$

Doing this repeatedly is called **dead reckoning**.

Odometry by hand, with this robot's real dimensions

Suppose over one 50 Hz tick ($\Delta t = 0.02$ s) the left wheel spins at $\omega_L = 3.0$ rad/s and the right at $\omega_R = 5.0$ rad/s.

Step 1 — wheel surface speeds.

$$v_L = 0.08135 \times 3.0 = 0.24405 \text{ m/s}, \quad v_R = 0.08135 \times 5.0 = 0.40675 \text{ m/s}.$$

Step 2 — body speeds, from equation (2.1).

$$\begin{aligned} v &= \frac{0.40675 + 0.24405}{2} = \frac{0.6508}{2} = 0.3254 \text{ m/s}, \\ \omega &= \frac{0.40675 - 0.24405}{0.2937} = \frac{0.1627}{0.2937} = 0.5540 \text{ rad/s}. \end{aligned}$$

Step 3 — one tick of motion. Starting at $(x, y, \theta) = (0, 0, 0)$:

$$\theta_1 = 0 + 0.5540 \times 0.02 = 0.01108 \text{ rad } (0.635^\circ),$$

$$x_1 = 0 + 0.3254 \times 0.02 \times \cos 0 = 0.006\,508\text{ m}, \quad y_1 = 0.$$

Step 4 — why this goes wrong. Now suppose the wheel radius is really 0.0805 m, not 0.08135 m — a 1% error, completely invisible. Every v is 1% too big. After the robot has driven 20 m, dead reckoning reports 20.2 m: a 0.2 m error that will never be corrected, because equation (2.2) only ever *adds*. Errors accumulate and never cancel. This is why odom drifts and why we need everything that follows.

2.3.1 Why rotation is the enemy on this robot

ROMR’s casters are *fixed*, not swivelling. When the robot rotates in place, the casters cannot roll in the direction they are being dragged, so they *scrub* sideways across the floor — measured in this project at 180 mm to 220 mm per turn. The wheel encoders happily report that the drive wheels turned exactly as commanded, because they did. What they cannot report is that the whole chassis also slid.

That produces a distinctive signature, which this project measured directly:

Condition	e (m)	$\Delta\theta$ (°)	σ_θ (rad)
Immediately after seeding from ground truth	0.014	−0.1	0.128
After 8 s standing still	0.015	−0.1	0.128
After one 90° turn	0.159	−17.4	1.189
After turning back	0.577	−24.9	1.233
After 0.8 m forward	0.736	−36.5	1.105
After 0.8 m reverse	0.289	−26.8	0.883

Standing still costs nothing. Driving straight costs a little and actually pulls σ_θ back *down*. One rotation multiplies σ_θ by $1.189/0.128 = 9.3$.

2.4 The belief, and the Bayes filter

Because the robot cannot know its pose, it maintains a whole density over poses. Write the pose at time t as $\mathbf{x}_t = (x_t, y_t, \theta_t)$, the control (commanded motion) as u_t , the sensor reading as z_t , and the map as m . The **belief** is

$$\text{bel}(\mathbf{x}_t) = p(\mathbf{x}_t \mid z_{1:t}, u_{1:t}, m), \quad (2.3)$$

read as: *the density over where I might be, given every sensor reading and every command since I switched on, and given the map.*

The **Bayes filter** updates this belief in two alternating steps, forever.

2.4.1 Step 1 — Prediction (the robot moved)

Before the new sensor reading arrives, account for the commanded motion:

$$\overline{\text{bel}}(\mathbf{x}_t) = \int p(\mathbf{x}_t \mid u_t, \mathbf{x}_{t-1}) \text{bel}(\mathbf{x}_{t-1}) d\mathbf{x}_{t-1}. \quad (2.4)$$

This integral is worth reading slowly, because it is the heart of everything.

Equation (2.4) in plain words

“To work out how strongly I should believe I am now at pose $\mathbf{x}_t$, consider *every* pose $\mathbf{x}_{t-1}$ I might have been at a moment ago. For each one, multiply (a) how strongly I believed I was there, $\text{bel}(\mathbf{x}_{t-1})$, by (b) the chance that the commanded motion u_t would have carried me from there to here, $p(\mathbf{x}_t | u_t, \mathbf{x}_{t-1})$. Then add up all those contributions.”

The $\int \dots d\mathbf{x}_{t-1}$ is that “add up over every possible previous pose”. The term $p(\mathbf{x}_t | u_t, \mathbf{x}_{t-1})$ is the **motion model** — it is where wheel slip, caster scrub and encoder error are admitted. The consequence: because the motion model is uncertain, this step always makes the belief *broader*. Moving makes you less sure.

2.4.2 Step 2 — Correction (the laser saw something)

Now fold in the measurement, using Bayes' rule (1.4):

$$\text{bel}(\mathbf{x}_t) = \eta p(z_t | \mathbf{x}_t, m) \overline{\text{bel}}(\mathbf{x}_t), \quad \eta = \frac{1}{\int p(z_t | \mathbf{x}_t, m) \overline{\text{bel}}(\mathbf{x}_t) d\mathbf{x}_t}. \quad (2.5)$$

$p(z_t | \mathbf{x}_t, m)$ is the **measurement model** or **likelihood**: given the map, and supposing the robot were at $\mathbf{x}_t$, how plausible is the scan we just received? Poses that would have produced this scan get their belief multiplied up; poses that would not get theirs multiplied down.

η (eta) is just the normaliser from equation (1.1): divide by the total so the area comes back to 1. It carries no information.

The whole of localisation, in one sentence

Movement spreads the belief out; measurement pulls it back in. A robot localises successfully when measurement can pull in faster than movement spreads out. Every parameter this project tuned is a parameter that changes the balance of that race — and Part 5 identifies which side of the race each one sits on.

2.4.3 Why we cannot just compute this

Equations (2.4) and (2.5) are exact and complete. If we could evaluate them we would be finished. We cannot, for two reasons.

First, $\mathbf{x}$ is three-dimensional and continuous. To store $\text{bel}(\mathbf{x})$ on a grid fine enough to be useful — say 5 cm and 5° — over this project's $13.9 \text{ m} \times 11.6 \text{ m}$ building, you would need roughly $278 \times 232 \times 72 \approx 4.6$ million cells, updated at 20 Hz. That is possible but wasteful, since almost all of those cells hold essentially zero.

Second, the integral in equation (2.4) has no closed-form solution for a realistic motion model. There is no formula to write down.

So we approximate. The approximation is the subject of Part 3.

Monte Carlo: a belief made of dots

3.1 The idea: replace a curve with a crowd

Instead of storing the belief as a function, store it as a *crowd of guesses*. Each guess is a complete hypothesis about the robot’s pose — an (x, y, θ) triple — and it is called a **particle**. Where the true belief is high, we put many particles; where it is low, we put few.

Formally, the belief is represented by a set of J weighted particles:

$$\mathcal{X}_t = \left\{ \langle \mathbf{x}_t^{[j]}, w_t^{[j]} \rangle \right\}_{j=1, \dots, J}, \quad \sum_{j=1}^J w_t^{[j]} = 1. \quad (3.1)$$

The superscript $[j]$ labels which particle; $w^{[j]}$ is that particle’s **weight**, its share of the total belief.

In this project J is not fixed: it varies between 500 and 2000, for reasons explained in Part 4 §4.4.

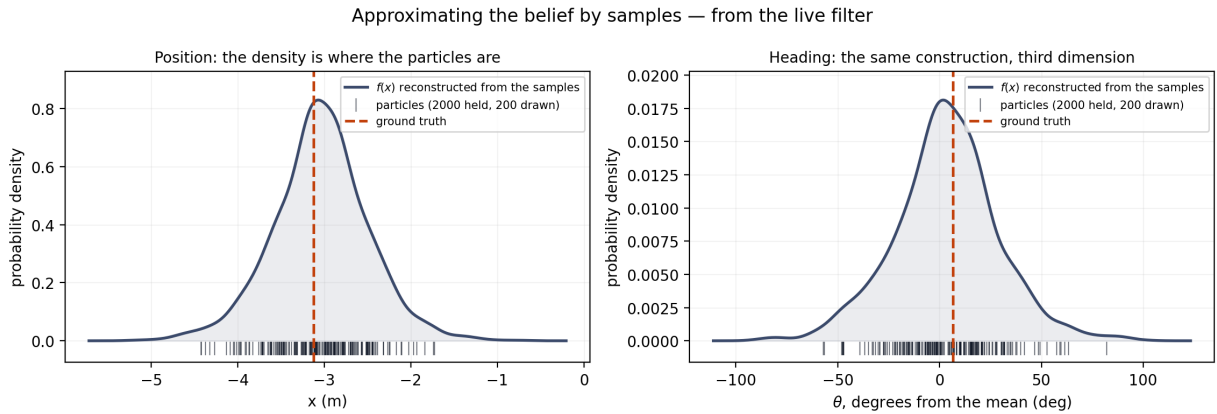


Figure 3.1: The same belief written two ways. Left: as a smooth density, the thing we cannot compute. Right: as a crowd of particles, the thing we can. Where the curve is tall, the dots are dense. This figure was generated for this project by `scripts/particle_approx_figure.py`.

3.2 Importance sampling: the mathematics that makes it legal

Why is a crowd of dots a legitimate stand-in for an integral? The answer is **importance sampling**, and it is worth doing properly because the whole evaluation strategy of this project descends from it.

3.2.1 The problem

We want an expectation — an average of some quantity $f(\mathbf{x})$ under the belief p :

$$\mathbb{E}_p[f] = \int f(\mathbf{x}) p(\mathbf{x}) d\mathbf{x}. \quad (3.2)$$

For example, if $f(\mathbf{x}) = x$, this gives the robot’s mean x coordinate — exactly the number AMCL publishes on `/amcl_pose`.

The trouble: we cannot draw samples from p . We do not have p in a form we can sample from; that is the whole difficulty.

3.2.2 The trick

Suppose there *is* some other density $\pi(\mathbf{x})$ that we *can* sample from — call it the **proposal**. Multiply and divide equation (3.2) by it:

$$\mathbb{E}_p[f] = \int f(\mathbf{x}) p(\mathbf{x}) d\mathbf{x} = \int f(\mathbf{x}) \frac{p(\mathbf{x})}{\pi(\mathbf{x})} \pi(\mathbf{x}) d\mathbf{x} = \mathbb{E}_\pi \left[f(\mathbf{x}) \frac{p(\mathbf{x})}{\pi(\mathbf{x})} \right]. \quad (3.3)$$

Nothing has been assumed and nothing has been lost — we multiplied by $\pi/\pi = 1$. But the meaning has changed completely. The left-hand side is an average under a density we cannot sample. The right-hand side is an average under a density we *can* sample, of a slightly different quantity.

Define the **importance weight**

$$w(\mathbf{x}) = \frac{p(\mathbf{x})}{\pi(\mathbf{x})} = \frac{\text{how much belief truly belongs here}}{\text{how often we actually generate samples here}}. \quad (3.4)$$

Then draw J samples $\mathbf{x}^{[j]} \sim \pi$ and estimate

$$\mathbb{E}_p[f] \approx \frac{\sum_{j=1}^J w(\mathbf{x}^{[j]}) f(\mathbf{x}^{[j]})}{\sum_{j=1}^J w(\mathbf{x}^{[j]})}. \quad (3.5)$$

(The division by $\sum w$ is the self-normalising form; it is what lets us get away with knowing p only up to a constant, which is all equation (2.5) ever gives us, because η is unknown.)

The one-sentence version

Sample from wherever is convenient, then correct for having sampled from the wrong place by weighting each sample by how wrong the place was.

3.2.3 The precondition nobody mentions, and why it matters here

Equation (3.3) requires that

$$p(\mathbf{x}) > 0 \implies \pi(\mathbf{x}) > 0. \quad (3.6)$$

The proposal must be able to generate samples *everywhere the truth might be*. If π is zero somewhere that p is not, the weight p/π becomes division by zero — and in practice, the estimator simply never sees that region and reports with total confidence about a place the robot is not.

This is precisely the failure that was measured

In the initial run, the robot’s belief converged tightly (σ small, e_{AMCL} typically under 0.4 m) onto a pose that was up to 15.66 m from the truth. Not one particle was near the real robot. The estimator was not “noisy” — it was *confidently answering a different question*, exactly as equation (3.6) warns. No amount of averaging fixes a violated support condition. This is why the corrected system addresses the *model*, not the number of particles.

3.3 The three steps of a particle filter

Each cycle of the filter does prediction, correction, and one extra step that has no counterpart in the exact Bayes filter.

The algorithm

Given the previous particle set $\mathcal{X}_{t-1}$, the control u_t , the scan z_t and the map m :

Step 1 — Sample (prediction). For each particle j :

$$\mathbf{x}_t^{[j]} \sim p(\mathbf{x}_t | u_t, \mathbf{x}_{t-1}^{[j]}).$$

Move every particle according to the odometry, plus a random perturbation drawn from the motion model. This is the Monte Carlo version of the integral in equation (2.4): instead of integrating over all previous poses, we push each of our J samples through the motion.

Step 2 — Weight (correction). For each particle j :

$$w_t^{[j]} = p(z_t | \mathbf{x}_t^{[j]}, m), \quad \text{then normalise} \quad w_t^{[j]} \leftarrow \frac{w_t^{[j]}}{\sum_k w_t^{[k]}}.$$

Ask each particle: “if the robot really were at your pose, how likely is the scan we just received?” That number is its weight. This is equation (2.5), and the normalisation is η .

Step 3 — Resample. Draw J new particles from the current set, *with replacement*, choosing particle j with probability $w^{[j]}$. Set all new weights to $1/J$.

3.3.1 Why Step 3 is necessary

Without resampling, after a few dozen cycles one particle holds a weight of essentially 1 and all the others hold essentially 0. You are then paying for J particles and getting the statistical value of one. This is called **weight degeneracy**. Resampling kills off the low-weight particles and duplicates the high-weight ones, so that the *density of particles* carries the information that the *weights* used to carry.

Notice the exchange: before resampling, belief lives in the weights; after resampling, all weights are equal and belief lives in where the particles are.

Resampling introduces its own disease

Duplicating particles destroys diversity. Run resampling often enough with no new information and every particle becomes a copy of one ancestor — this is **particle depletion** or **sample impoverishment**. If the true pose was not among the survivors, it can never be recovered, because Step 1 only ever jitters existing particles; it never invents new ones far away.

This project sets `resample_interval: 1`, meaning it resamples on *every* filter update. That is the most aggressive setting and it makes depletion a live concern. It is one of the tuning levers identified in Part 11.

3.4 Seeing it happen on this robot

The following four figures were produced from this project’s own filter, using `scripts/particle_figure.py`, and they show the mechanism directly.

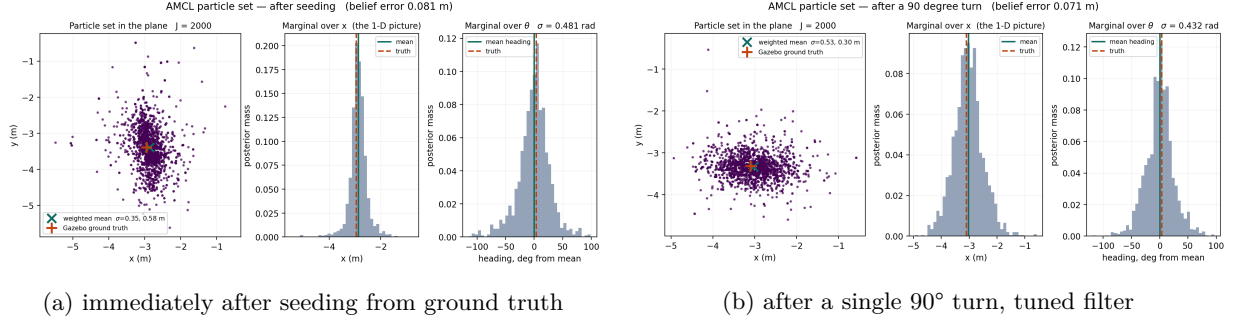


Figure 3.2: The particle cloud with the corrected parameters. Seeding places a tight cluster on the true pose. One rotation spreads it — prediction always spreads — but the scan correction holds it together.

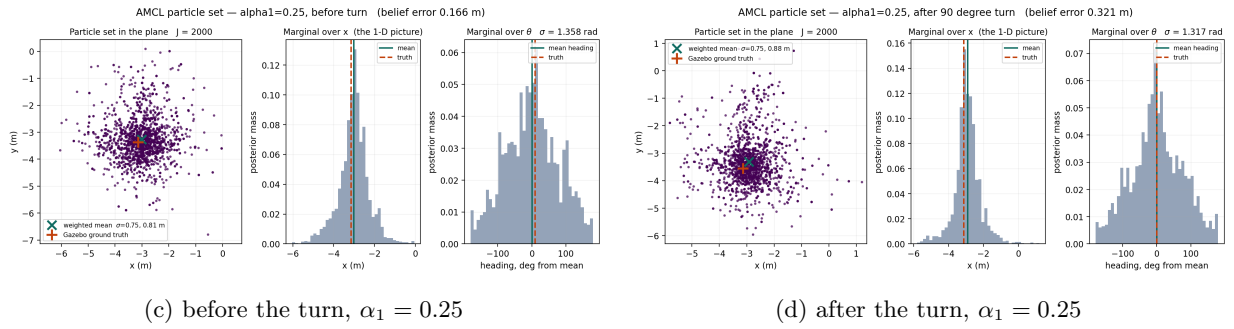


Figure 3.3: The same rotation with the rotational noise coefficient set to 0.25. The prediction step sprays particles across a wide arc, faster than one scan update can pull them back. Panels (c)–(d) are the visual form of the sweep in Part 5 §5.3.

Resampling: the wheel, and the better wheel

Step 3 of the filter said “draw J new particles, choosing particle j with probability $w^{[j]}$ ”. That sentence hides a choice of algorithm, and the two main candidates behave very differently. This part explains both, works each one by hand on the same five particles, proves the guarantee that separates them, and then establishes — from the actual source code of the software this project runs — which one is in use here and why.

4.1 The cumulative distribution: turning weights into a ruler

Both algorithms start the same way. Lay the weights end to end along a line of total length 1. Particle j occupies a segment of length $w^{[j]}$. Formally we build the **cumulative distribution** c :

$$c_0 = 0, \quad c_i = \sum_{j=1}^i w^{[j]} \quad \text{for } i = 1, \dots, J, \quad c_J = 1. \quad (4.1)$$

Now “choosing particle j with probability $w^{[j]}$ ” becomes something mechanical: pick a number r somewhere in $[0, 1)$, find which segment it lands in — the i with $c_{i-1} \leq r < c_i$ — and take that particle. Because segment i has length $w^{[i]}$, a uniformly random r lands in it with probability exactly $w^{[i]}$. That is the requirement, met exactly.

Our running example

Five particles with these weights (they sum to 1):

particle j	1	2	3	4	5
weight $w^{[j]}$	0.05	0.35	0.10	0.42	0.08
segment	$[0, 0.05)$	$[0.05, 0.40)$	$[0.40, 0.50)$	$[0.50, 0.92)$	$[0.92, 1.00)$

Cumulative distribution: $c = (0, 0.05, 0.40, 0.50, 0.92, 1.00)$.

With $J = 5$, the number of copies we *ought* to get of each particle is $Jw^{[j]}$:

$$(0.25, 1.75, 0.50, 2.10, 0.40).$$

Since we can only take whole particles, no algorithm can hit these exactly. The question is how badly each one misses.

4.2 Method A — the roulette wheel (multinomial resampling)

Bend the line of segments into a circle and fit a single pointer. Spin the pointer J times, independently, and take whichever particle it lands on each time.

Algorithm A: roulette wheel / multinomial

1. Build c by equation (4.1).
2. Repeat J times:
 - a. Draw $r \sim \text{Uniform}[0, 1)$ — a *fresh* random number every time.
 - b. Find i such that $c_{i-1} \leq r < c_i$.
 - c. Copy particle i into the new set.

Method A worked by hand

Take these five random numbers (any five would do): $r = 0.13, 0.68, 0.91, 0.07, 0.55$.

r	lands in segment	selects
0.13	[0.05, 0.40)	particle 2
0.68	[0.50, 0.92)	particle 4
0.91	[0.50, 0.92)	particle 4
0.07	[0.05, 0.40)	particle 2
0.55	[0.50, 0.92)	particle 4

Result: counts (0, 2, 0, 3, 0) against the ideal (0.25, 1.75, 0.50, 2.10, 0.40).

Particle 3 carried 10% of the belief and was wiped out. Particle 5 carried 8% and was wiped out. Nothing went wrong — this is a perfectly valid draw — but a tenth of the filter's information vanished on this one step.

4.2.1 How much does it miss by? (exact answer)

Because each of the J draws is independent and each hits particle j with probability $w^{[j]}$, the count N_j of copies of particle j is a **binomial** random variable:

$$N_j \sim \text{Binomial}(J, w^{[j]}), \quad \mathbb{E}[N_j] = Jw^{[j]}, \quad \text{Var}[N_j] = Jw^{[j]}(1 - w^{[j]}). \quad (4.2)$$

The mean is right — the method is *unbiased* — but the variance grows with J . This is the flaw.

4.3 Method B — stochastic universal sampling (systematic, low variance)

Keep the same wheel. Replace the single pointer with a *comb* of J pointers, rigidly spaced exactly $1/J$ apart. Spin the comb *once*, and read off all J pointers together.

Algorithm B: stochastic universal sampling

1. Build c by equation (4.1).
2. Draw *one* random offset $u \sim \text{Uniform}[0, 1/J)$.
3. For $n = 0, 1, \dots, J-1$: set the pointer $r_n = u + \frac{n}{J}$ and select the particle whose segment contains r_n .

Only one random number is drawn for the entire step.

Method B worked by hand, same particles

$J = 5$, so pointers are spaced $1/5 = 0.20$ apart. Draw one offset $u = 0.13$ (it must lie in $[0, 0.20)$). The comb is then

$$r_0 = 0.13, \quad r_1 = 0.33, \quad r_2 = 0.53, \quad r_3 = 0.73, \quad r_4 = 0.93.$$

pointer	lands in segment	selects
0.13	$[0.05, 0.40)$	particle 2
0.33	$[0.05, 0.40)$	particle 2
0.53	$[0.50, 0.92)$	particle 4
0.73	$[0.50, 0.92)$	particle 4
0.93	$[0.92, 1.00)$	particle 5

Result: counts $(0, 2, 0, 2, 1)$ against the ideal $(0.25, 1.75, 0.50, 2.10, 0.40)$.

Compare with Method A's $(0, 2, 0, 3, 0)$. Method B kept particle 5; Method A did not. And notice that particle 4, whose ideal count is 2.10, received exactly 2 — it could only ever have received 2 or 3, never 0 and never 5.

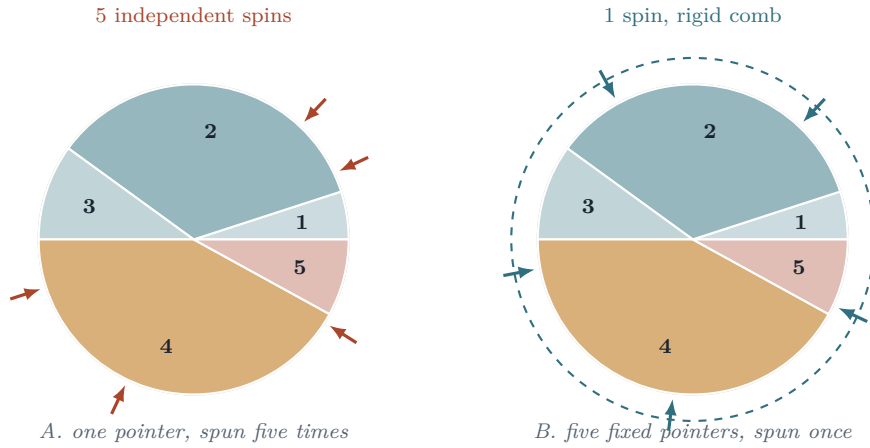


Figure 4.1: The same weighted wheel, two ways of reading it. Segment sizes are the weights $(0.05, 0.35, 0.10, 0.42, 0.08)$. Left: the roulette wheel is spun five times independently; the pointers clump, and segments 3 and 5 are missed entirely. Right: five pointers rigidly spaced $1/5$ of a turn apart are placed by a single spin; they cannot clump, so any segment wider than $1/5$ of the wheel is certain to be hit.

4.3.1 The guarantee, proved

Survival guarantee of stochastic universal sampling

Under Method B, *every particle whose weight satisfies $w^{[j]} \geq 1/J$ is selected at least once, with certainty.*

Proof. Particle j occupies the half-open interval $[c_{j-1}, c_j)$ of length $w^{[j]}$. The pointers form the arithmetic sequence $u, u + 1/J, u + 2/J, \dots$, and consecutive pointers are exactly $1/J$ apart. Any interval of length $\ell \geq 1/J$ placed anywhere on $[0, 1)$ must contain at least one point of a lattice of spacing $1/J$: if it contained none, there would be a gap of length greater than $1/J$ between two consecutive lattice points, contradicting the spacing. Hence $w^{[j]} \geq 1/J$ implies at least one

pointer lands in particle j 's segment. ■

Method A offers no such guarantee. Under Method A the probability that particle j is missed entirely is

$$P(N_j = 0) = (1 - w^{[j]})^J,$$

which is small for large J but never zero.

4.3.2 The variance, quantified

Under Method B the count N_j can only take one of two adjacent whole numbers. Write $Jw^{[j]} = \lfloor Jw^{[j]} \rfloor + f_j$, where f_j is the fractional part. Then

$$N_j \in \{\lfloor Jw^{[j]} \rfloor, \lceil Jw^{[j]} \rceil\}, \quad \mathbb{E}[N_j] = Jw^{[j]}, \quad \text{Var}[N_j] = f_j(1 - f_j) \leq \frac{1}{4}. \quad (4.3)$$

The variance is bounded by $1/4$ *regardless of J* . Compare with equation (4.2), where the variance grows linearly in J .

Putting numbers on the difference

Particle 4 in our example: $w = 0.42$, $J = 5$, so $Jw = 2.10$ and $f = 0.10$.

Method A (multinomial).

$$\text{Var}[N_4] = Jw(1 - w) = 5 \times 0.42 \times 0.58 = 1.218, \quad \sigma = \sqrt{1.218} = 1.10.$$

So the count typically wanders by about one whole particle either side of 2.1.

Method B (systematic).

$$\text{Var}[N_4] = f(1 - f) = 0.10 \times 0.90 = 0.09, \quad \sigma = \sqrt{0.09} = 0.30.$$

Ratio: $1.218/0.09 = 13.5$. Method B has roughly **thirteen times less** count variance on this particle.

Now scale up to this project's actual particle count. With $J = 2000$ and a particle holding $w = 0.001$:

$$\text{A: Var} = 2000 \times 0.001 \times 0.999 = 1.998 \quad \text{vs.} \quad \text{B: Var} \leq 0.25.$$

And the gap widens as J grows, because A's variance is proportional to J and B's is not.

4.3.3 Running cost

Method	Random numbers	Search cost for J draws
A, linear scan of c	J	$O(J^2)$ worst case
A, binary search on c	J	$O(J \log J)$
B, systematic	1	$O(J)$ — one sweep, pointers are sorted

Method B is cheaper because the pointers arrive in increasing order, so a single walk through c serves all of them; no search is needed at all.

4.4 Which one does this project actually use, and why?

An honest correction to the framing of the question

This was not a choice made in this thesis. The resampler is inside `nav2_amcl`, which this project uses as supplied; there is no parameter that switches it. The defensible statement is therefore not “we chose A over B” but “*AMCL* uses A, for a documented reason, and that reason interacts with a parameter this project *did* set.”

The implementation is `nav2_amcl/src/pf/pf.c`, function `pf_update_resample`. On the `humble` branch — the branch this project runs — it builds the cumulative distribution

```
c[0] = 0.0;
for (i = 0; i < set_a->sample_count; i++) {
    c[i + 1] = c[i] + set_a->samples[i].weight;
}
```

and then, for each new particle, draws a fresh random number and scans for its segment:

```
double r;
r = drand48();
for (i = 0; i < set_a->sample_count; i++) {
    if ((c[i] <= r) && (r < c[i + 1])) {
        break;
    }
}
```

That is Method A exactly: one independent `drand48()` per particle, and a linear scan rather than a binary search.

The same file contains a commented-out low-variance sampler and the reason it was not used:

Verbatim comment in `nav2_amcl/src/pf/pf.c`

“Can’t (easily) combine low-variance sampler with KLD adaptive sampling, so we’ll take the more traditional route.”

4.4.1 What KLD adaptive sampling is, and why it blocks Method B

The “A” in AMCL stands for **adaptive**, and what adapts is J itself. Rather than always carrying a fixed number of particles, AMCL carries as few as it can get away with: many when it is lost and the belief is spread over the whole building, few when it has converged onto one spot.

The rule it uses is **KLD sampling**. The filter drops particles into a k -d tree and counts k , the number of distinct occupied bins — a proxy for how spread out the belief is. It then computes the number of particles n needed so that the error between the particle approximation and the true distribution, measured as a Kullback–Leibler divergence, exceeds ε with probability at most δ :

$$n = \left\lceil \frac{k-1}{2\varepsilon} \left(1 - \frac{2}{9(k-1)} + \sqrt{\frac{2}{9(k-1)}} z_{1-\delta} \right)^3 \right\rceil, \quad (4.4)$$

then clamps n into `[min_particles, max_particles]`.

The terms:

- k — occupied bins in the k -d tree. Small k = converged belief.

- ε — the tolerated KL error. In nav2’s source, `pf->pop_err = 0.01`.
- $z_{1-\delta}$ — the Gaussian quantile from Part 1 §1.4. In nav2’s source, `pf->pop_z = 3`, i.e. the “three sigma” level, $\delta \approx 0.001$.
- $\lceil \cdot \rceil$ — round up to a whole number of particles.

Equation (4.4) by hand, with this project’s limits

This project sets `min_particles: 500` and `max_particles: 2000`. Take $k = 5$ occupied bins (a fairly converged cloud).

Step 1. $k - 1 = 4$, so $9(k - 1) = 36$.

$$\frac{2}{9(k - 1)} = \frac{2}{36} = 0.055556.$$

Step 2. $\sqrt{0.055556} = 0.235702$, and multiplying by $z = 3$:

$$0.235702 \times 3 = 0.707107.$$

Step 3. The bracket:

$$1 - 0.055556 + 0.707107 = 1.651551.$$

Step 4. Cube it:

$$1.651551^3 = 4.5048.$$

Step 5. Multiply by $\frac{k - 1}{2\varepsilon} = \frac{4}{0.02} = 200$:

$$200 \times 4.5048 = 900.96 \rightarrow \lceil 900.96 \rceil = 901.$$

Step 6. Clamp to $[500, 2000]$: $n = \mathbf{901}$ particles.

Repeat for $k = 2$ (a very tight cloud): $2/9 = 0.222222$; $\sqrt{0.222222} \times 3 = 1.414214$; bracket $= 1 - 0.222222 + 1.414214 = 2.191992$; cubed $= 10.5321$; $\times 1/0.02 = 50$ gives $526.6 \rightarrow 527$. Clamped: **527**.

And $k = 20$ (a badly spread cloud): $2/171 = 0.011696$; $\sqrt{\cdot} \times 3 = 0.324443$; bracket $= 1.312747$; cubed $= 2.2623$; $\times 19/0.02 = 950$ gives $2149.2 \rightarrow 2150$, which exceeds `max_particles` and is clamped to **2000**.

So in this project the filter runs between 527 and 2000 particles depending on how uncertain it is — and the floor of 500 almost never binds, because $k = 2$ already asks for 527.

4.4.2 The actual incompatibility

Method B needs to know J *before* it starts, because the pointer spacing is $1/J$ and the offset must be drawn from $[0, 1/J)$. But KLD sampling decides J *while* resampling is in progress: nav2 draws particles one at a time, inserts each into the k -d tree, recomputes the limit from equation (4.4), and stops when it has enough:

```
if (set_b->sample_count > pf_resample_limit(pf,
set_b->kdtree->leaf_count)) {
    break;
}
```

The target count is a moving quantity that depends on the particles drawn so far. A rigid comb of $1/J$ -spaced pointers cannot be laid down against a J that is not yet known. That is the incompatibility the source comment refers to, stated concretely.

4.4.3 What this means for this project's results

The defensible position

1. AMCL uses multinomial (roulette-wheel) resampling. This is verifiable in `nav2_amcl/src/pf/pf.c` and is not configurable.
2. It does so in order to keep KLD adaptive sampling, which this project relies on — `min_particles: 500, max_particles: 2000` are meaningless without it.
3. The price is higher count variance and no survival guarantee, so **particle depletion** is bounded only probabilistically, not absolutely.
4. This project sets `resample_interval: 1` — resample on every update — which is the setting that most exposes that price. Raising it is a tuning lever, listed in Part 11.
5. The mitigation actually used here was not to change the resampler but to *reduce the need for it*: sharpening the measurement likelihood (`z_hit` $0.5 \rightarrow 0.8$, `laser_max_range` $8 \rightarrow 16$ m, `max_beams` $60 \rightarrow 120$) so that weights are informative, and narrowing the motion model ($\alpha_1 = \alpha_2 : 0.25 \rightarrow 0.02$) so that fewer particles are thrown into low-weight regions in the first place. Both are covered in Part 5.

AMCL: every knob, and what it does

AMCL stands for *Adaptive Monte Carlo Localisation*. Taking the words backwards: *Localisation* — working out where the robot is; *Monte Carlo* — doing it with the crowd of particles from Part 3; *Adaptive* — varying the size of the crowd by the KLD rule of Part 4 §4.4.

This part goes through the configuration used in this project, `config/nav2_params.yaml`, and says what each parameter physically does. It is organised by the split that matters — proposal versus weight.

5.1 The split: what you choose, and what chooses you

Return to importance sampling, equation (3.3). There are exactly two ingredients: the proposal π you sample from, and the weight $w = p/\pi$ you correct with. They are not symmetric.

The proposal is a design choice; the weight is not

π is *yours*. Any π satisfying the support condition (3.6) gives a correct estimator; a bad π makes it *inefficient*, needing more particles for the same accuracy, but it still converges on the right answer.

w is *forced*. Once you have chosen π , the weight must be p/π exactly. Get it wrong and the ratio no longer cancels in equation (3.3): you are now computing an average under some other density altogether. A weight fault does not make the estimator slow — it makes it answer a different question, confidently.

In AMCL:

- the **proposal** is the **motion model**, governed by $\alpha_1 \dots \alpha_5$;
- the **weight** is the **measurement model**, governed by `z_hit`, `z_rand`, `z_short`, `z_max`, `sigma_hit`, `max_beams` and `laser_max_range`.

Answering the question directly: what did this project choose?

This project chose the **proposal**. Specifically it chose $\alpha_1 = \alpha_2 = 0.02$ and $\alpha_3 = \alpha_4 = \alpha_5 = 0.05$, by sweeping and measuring rather than by reasoning (§5.3), and it chose to disable divergence recovery, `recovery_alpha_slow = recovery_alpha_fast = 0` (§5.5).

It also **corrected the weight** — but that is a different kind of act. The scanner fitted to ROMR reaches 16m and `laser_max_range` was set to 8m. That is not a preference; it is a description of the sensor that was factually false, and it was therefore a *fault*, not a tuning. Correcting it to 16m restored the correspondence between w and p/π . The same applies to raising `z_hit` from 0.5 to 0.8: the value 0.5 asserted that half of every scan is meaningless noise, and the measurement in §5.4 shows that against this map it is not.

5.2 The proposal: the odometry motion model and the five alphas

At each step, AMCL takes the change in odometry and decomposes it into three elementary movements: an initial rotation, a translation, and a final rotation.

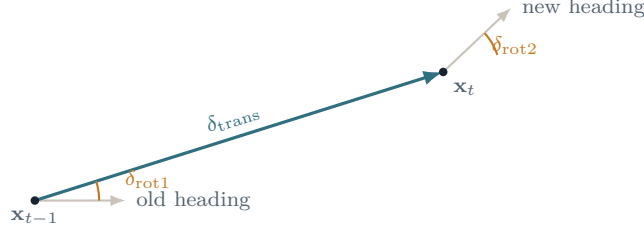


Figure 5.1: Any change of pose is written as: turn on the spot to face the destination (δ_{rot1}), drive straight (δ_{trans}), turn on the spot to the final heading (δ_{rot2}). The five α coefficients say how much random error to add to each of these three, and where that error comes from.

The decomposition is

$$\begin{aligned}\delta_{\text{rot1}} &= \text{atan2}(\bar{y}' - \bar{y}, \bar{x}' - \bar{x}) - \bar{\theta}, \\ \delta_{\text{trans}} &= \sqrt{(\bar{x}' - \bar{x})^2 + (\bar{y}' - \bar{y})^2}, \\ \delta_{\text{rot2}} &= \bar{\theta}' - \bar{\theta} - \delta_{\text{rot1}},\end{aligned}\tag{5.1}$$

where barred quantities are what the *odometry* reports (not the truth).

AMCL then perturbs each of the three with zero-mean noise whose variance is built from the alphas:

$$\begin{aligned}\hat{\delta}_{\text{rot1}} &= \delta_{\text{rot1}} - \mathcal{E}(\alpha_1 \delta_{\text{rot1}}^2 + \alpha_2 \delta_{\text{trans}}^2), \\ \hat{\delta}_{\text{trans}} &= \delta_{\text{trans}} - \mathcal{E}(\alpha_3 \delta_{\text{trans}}^2 + \alpha_4 (\delta_{\text{rot1}}^2 + \delta_{\text{rot2}}^2)), \\ \hat{\delta}_{\text{rot2}} &= \delta_{\text{rot2}} - \mathcal{E}(\alpha_1 \delta_{\text{rot2}}^2 + \alpha_2 \delta_{\text{trans}}^2),\end{aligned}\tag{5.2}$$

where $\mathcal{E}(b)$ means “a random draw with mean zero and variance b ”. Each particle gets its own independent draw, which is what makes the cloud spread.

Parameter	This project	What it means
α_1	0.02	rotational error caused by rotating
α_2	0.02	rotational error caused by translating
α_3	0.05	translational error caused by translating
α_4	0.05	translational error caused by rotating
α_5	0.05	extra term used by omnidirectional models; inert for a differential-drive robot

Read the units carefully — this is where the mistake was made

Look at equation (5.2). The alphas multiply the *square of the motion*, and the result is a *variance*. So α_1 is not “how much I distrust rotation” on some abstract scale. It is the constant of proportionality between the size of a turn and the variance of the error attributed to that turn. Doubling α_1 doubles the variance, which multiplies the standard deviation — the physical spread of the cloud — by $\sqrt{2}$.

5.3 The tuning that went backwards, and what it teaches

ROMR’s casters are fixed and scrub 180–220 mm per turn. The obvious inference: rotational odometry is unreliable, therefore α_1 should be *raised* so the filter trusts it less. That was tried, at $\alpha_1 = 0.25$. It is the worst setting on the curve.

Sweeping $\alpha_1 = \alpha_2$, re-seeding from ground truth before each trial, and applying one 90° turn:

$\alpha_1 = \alpha_2$	e (m)	$\Delta\theta$ (°)	σ_θ (rad)
0.25	0.469	-13.3	1.257
0.10	0.279	-9.5	0.958
0.05	0.168	+1.0	0.694
0.02	0.108	-2.8	0.441
0.01	0.104	-1.8	0.364

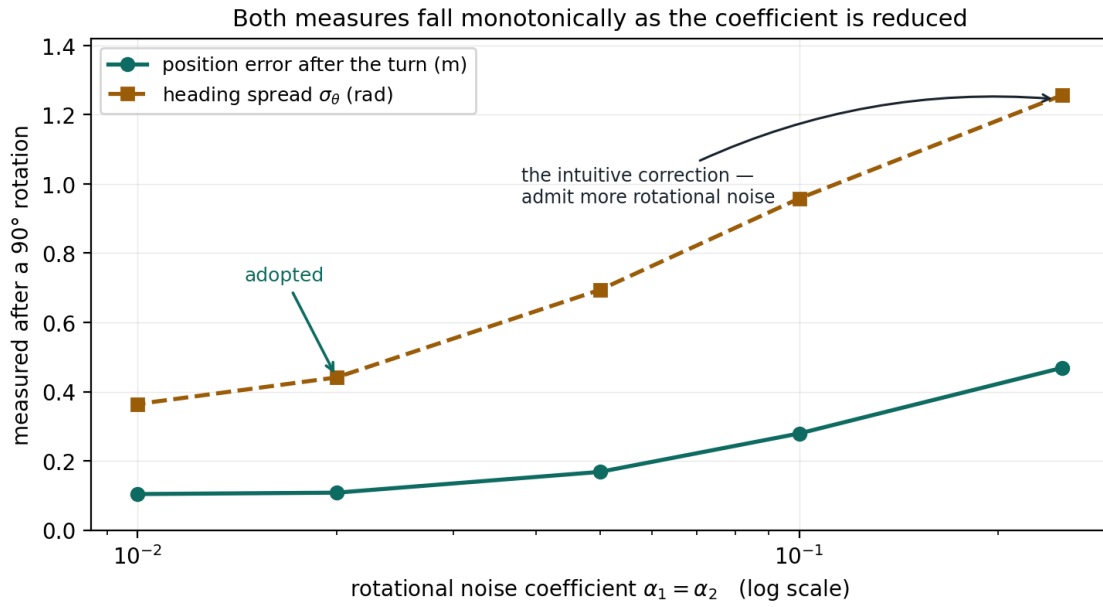


Figure 5.2: Position error and heading standard deviation after a single 90° rotation, against the rotational noise coefficient. Monotonic and decreasing across the swept range, flattening below 0.02. Regenerate with `scripts/plot_results.py`.

Why raising α_1 made it worse — the race, again

Part 2 §2.4 put it as a race: prediction spreads the belief, correction pulls it in. Raising α_1 makes the prediction step spray particles across a wide arc. The arc grows faster than one scan update can pull it back, so the next prediction starts from an already-spread cloud and sprays wider still. The filter loses the race.

The correct reading of the caster scrub is not “add noise”. It is: *the scrub is a systematic error, not a random one*. It biases the odometry the same way every time. Random noise in the motion model cannot represent a systematic bias; all it does is degrade the proposal. The systematic part has to be handled by the measurement, and the measurement can only handle it if it is sharp enough — which is what §5.4 is about.

Why 0.02 and not 0.01, when 0.01 measured marginally better? Because a filter that believes its odometry perfectly has no mechanism to accept a correction when the odometry is finally wrong. The scrub is real. 0.02 keeps enough genuine rotational noise in the model to leave the correction step something to work with, at a cost of 0.004m in the measured error. That is a judgement, and it is stated as one.

5.4 The weight: the measurement model

The laser publishes a scan $\mathbf{z} = \{z_1, \dots, z_K\}$ of range readings. For a candidate pose $\mathbf{x}$ and the map m , AMCL scores it by equation (1.5). Each individual beam likelihood is a mixture of four things that could have produced that reading:

$$p(z_k | \mathbf{x}, m) = z_{\text{hit}} p_{\text{hit}} + z_{\text{short}} p_{\text{short}} + z_{\text{max}} p_{\text{max}} + z_{\text{rand}} p_{\text{rand}}, \quad \sum z_{\bullet} = 1. \quad (5.3)$$

Component	This project	What it models
z_{hit}	0.80	the beam hit the wall the map predicts; modelled as a Gaussian of width <code>sigma_hit</code> around the expected range
z_{short}	0.05	the beam hit something not in the map (a person, an object); exponential, rate <code>lambda_short</code>
z_{max}	0.05	the beam hit nothing and returned max range
z_{rand}	0.20	the beam is nonsense; uniform over the range

The dominant term is the first. Writing $\hat{z}_k$ for the range the map says *ought* to be measured from pose $\mathbf{x}$ along beam k :

$$p_{\text{hit}}(z_k | \mathbf{x}, m) = \frac{1}{\sqrt{2\pi} \sigma_{\text{hit}}^2} \exp\left(-\frac{(z_k - \hat{z}_k)^2}{2 \sigma_{\text{hit}}^2}\right), \quad (5.4)$$

which is exactly the Gaussian of equation (1.3), centred on the predicted range. In this project $\sigma_{\text{hit}} = 0.2$ m: a beam within 20 cm of where the map says the wall is counts as a good match.

Why $z_{\text{hit}} + z_{\text{rand}} = 0.5/0.5$ was wrong here

Setting $z_{\text{hit}} = z_{\text{rand}} = 0.5$ asserts that half of every scan is meaningless. Whether that is true is measurable, and it was measured. Taking a scan at the *true* pose and asking how far each of the 720 beams falls from the nearest wall in the map:

Mean distance to nearest wall	0.047 m
Median distance	0.050 m
Within 0.10 m	66.8%
Within 0.20 m	100%

Every single beam agrees with the map to within 20 cm. In this environment there is essentially no noise to model. Weighting the informative term at 0.8 rather than 0.5 sharpens the likelihood peak, which is exactly what wins the race in §5.3.

5.4.1 The other two weight faults

laser_max_range: 8 → 16 metres. The fitted scanner reaches 16 m; the building is 13.85 m across. An 8 m cutoff discards exactly the long returns that pin a pose down along a corridor. A robot in a featureless hallway with only short beams available has no way to tell where along the hallway it is; the likelihood becomes flat in one direction, and the belief slides.

max_beams: 60 → 120. Of the 720 beams available, 60 leaves the likelihood peak shallower than the cost justifies. 120 deepens it. Note the tension with the independence caveat of Part 1 §1.6: more beams means a sharper peak, but also more over-confidence from treating correlated beams as independent. 120 of 720 — one beam every 3° — was the compromise adopted.

5.5 Divergence recovery, and why it is off here

AMCL can inject random particles across the map when it suspects it is lost. It decides this by comparing a fast-moving and a slow-moving average of the total particle weight; if the fast average drops well below the slow one, the scan has stopped matching and something is wrong. The rate constants are `recovery_alpha_fast` and `recovery_alpha_slow`, and the widely quoted values are 0.1 and 0.001.

This project sets both to **0**, disabling the feature. That is unusual, and the reasoning is worth spelling out.

The argument for switching it off, in three steps

- 1. It measurably made things worse.** With recovery enabled, σ_θ reached 2.53 rad — close to uniform over a full circle — while the robot sat 8.5 m from where AMCL said it was.
- 2. The mechanism is clear.** The pose AMCL publishes is the mean over *all* particles. Injecting random particles across the map therefore drags the reported mean away from a cluster that may have been perfectly good. Recovery does not replace a bad answer with a better one; it dilutes whatever answer exists.
- 3. The precondition does not hold here.** Recovery earns its place when the environment is ambiguous — when a scan is genuinely consistent with two or more distinct poses, and the filter can back the wrong one. Measured against this map, that ambiguity does not exist. Rotating the assumed heading about the true pose and recomputing scan–map agreement gives 0.047 m at zero offset, against 0.443 m at 45°, 0.410 m at 90°, 0.762 m at 135° and 0.749 m at 180°. There is one sharp global minimum and no second hypothesis for recovery to find.

The cost of this decision, stated plainly

With recovery off, if localisation is ever genuinely lost, AMCL will *not* recover on its own. It must be re-seeded — by `scripts/seed.sh`, or by clicking **2D Pose Estimate** in RViz. This is an acceptable trade in a mapped, unambiguous, single-storey environment. It would not be acceptable in a repetitive one (a hotel corridor, a warehouse aisle), and a reviewer is entitled to say so.

5.6 The remaining parameters

Parameter	Value	Meaning
<code>min_particles</code>	500	floor on J from equation (4.4)
<code>max_particles</code>	2000	ceiling on J
<code>update_min_d</code>	0.25 m	do not run a filter update until the robot has moved this far; saves compute and avoids resampling on noise
<code>update_min_a</code>	0.2 rad	same, for rotation (11.5°)
<code>resample_interval</code>	1	resample on <i>every</i> update; see Part 11
<code>laser_min_range</code>	0.15 m	ignore returns from the robot's own body
<code>laser_max_range</code>	16 m	matches the fitted scanner
<code>sigma_hit</code>	0.2 m	width of the Gaussian in eq. (5.4)
<code>lambda_short</code>	0.1	decay rate of the unexpected-obstacle term
<code>transform_tolerance</code>	1.0 s	how long the published <code>map</code> → <code>odom</code> transform stays valid
<code>robot_model_type</code>	<code>DifferentialMotionModel</code>	selects eq. (5.2)
<code>tf_broadcast</code>	<code>true</code>	AMCL publishes the correction (Figure 2.1)
<code>set_initial_pose</code>	<code>false</code>	the pose is supplied externally, by RViz or <code>seed.sh</code>

5.7 What AMCL reports back, and the covariance

AMCL publishes on `/amcl_pose`, a `geometry_msgs/PoseWithCovarianceStamped`. Two things are in it.

The pose is the weighted mean over the particle set — the Monte Carlo version of equation (1.2):

$$\hat{\mathbf{x}} = \sum_{j=1}^J w^{[j]} \mathbf{x}^{[j]}. \quad (5.5)$$

The covariance is the spread about that mean:

$$\Sigma = \sum_{j=1}^J w^{[j]} (\mathbf{x}^{[j]} - \hat{\mathbf{x}})(\mathbf{x}^{[j]} - \hat{\mathbf{x}})^{\top}. \quad (5.6)$$

Σ is a 6×6 matrix. Its diagonal entries are the variances, and their square roots are the standard deviations this project logs as `sigma_x`, `sigma_y` and `sigma_yaw`. The validation script combines the first two into

$$\sigma_{\text{pos}} = \sqrt{\sigma_x^2 + \sigma_y^2}.$$

The covariance is the filter's opinion of itself

Σ measures how much the *particles* disagree with each other. It does not measure how far they are from the truth. In the initial run the particles agreed closely with one another — small σ — while sitting 15 m from the robot. A confident filter and a correct filter are different things, and the whole evaluation design in Part 7 exists because of that distinction.

Nav2: the stack around the filter

6.1 What Nav2 is

Nav2 (Navigation 2) is the ROS 2 navigation stack. You hand it a goal pose; it plans a route, drives the robot along it, recovers when it gets stuck, and eventually reports an outcome. It is built as a set of independent **servers**, each doing one job, coordinated by a **behaviour tree**.

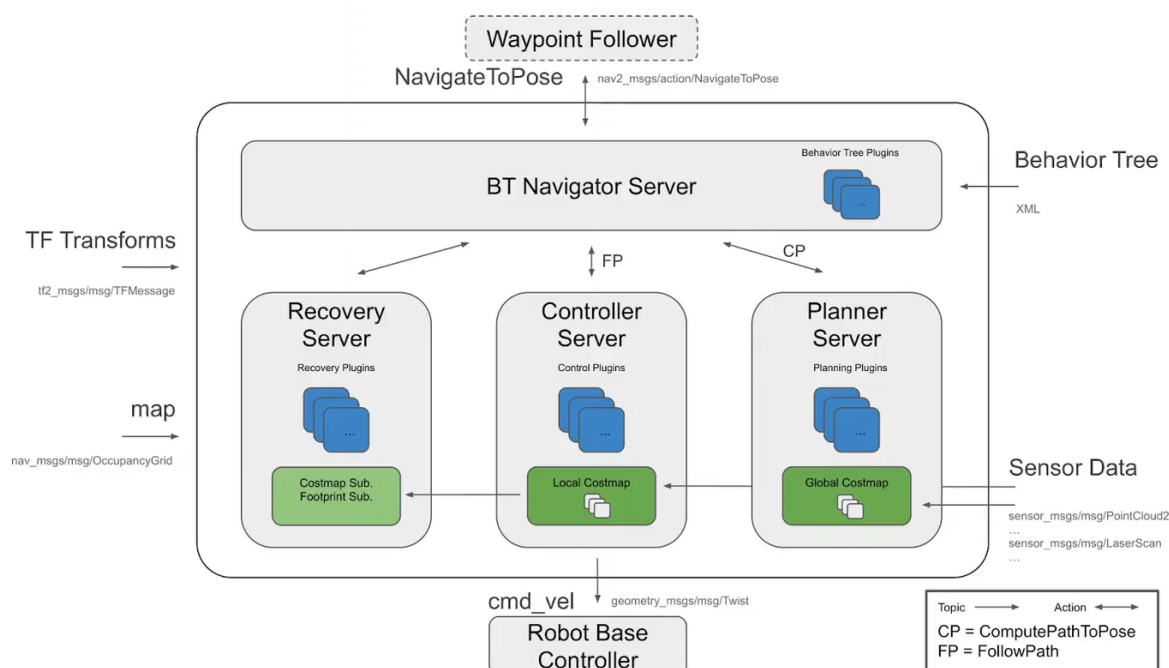


Figure 6.1: The Navigation 2 architecture, reproduced from the Navigation 2 documentation. A goal enters as a **NavigateToPose** action at the BT Navigator Server, which calls the Planner, Controller and Recovery Servers through further actions (**CP** = **ComputePathToPose**, **FP** = **FollowPath**). The output is a stream of `cmd_vel` velocity commands to the base controller. Single arrows are topics; double arrows are actions. *This figure is not original work and should be cited to its source wherever it is used.*

6.1.1 Server by server

BT Navigator Server

The coordinator. It runs a behaviour tree — an XML file describing what to try, in what order, and what to do on failure. It accepts the **NavigateToPose** action and is the only server the outside world talks to.

Planner Server

Given a start and a goal, produces a path. This project uses **NavFn** under the plugin name **GridBased**: a grid search over the global costmap, described in detail in §6.1.2. It answers the **ComputePathToPose** (CP) action. When this project logged “failed to create plan”, it was this server.

Controller Server

Given a path, produces velocity commands. This project uses `DWBLocalPlanner` (Dynamic Window Approach): at every one of its 20 Hz cycles it simulates a fan of candidate trajectories 1.5 s into the future, scores each with a set of **critics** — `PathAlign`, `GoalAlign`, `BaseObstacle`, `RotateToGoal`, `Oscillation`, `PathDist` — and issues the best. It answers the `FollowPath` (FP) action. It also hosts the goal checker and the progress checker, which §6.4 shows to be the source of this project’s remaining defect.

Recovery Server

What to do when stuck: spin in place, back up, wait, clear the costmaps. This is the server that produced the sideways shuffling observed at the garage.

Costmaps

Two occupancy grids. The **global costmap** covers the whole building and is what the planner searches. The **local costmap** is a small rolling window around the robot and is what the controller uses to avoid things that have just appeared.

6.1.2 Which planning algorithm — and why not RRT

Path planners split into two families, and it is worth knowing which one you are using and why.

Search-based methods treat the map as a grid of cells and examine them systematically. **Dijkstra’s algorithm** expands outward from the goal like a flood, one ring of cells at a time, recording for every cell how far it is from the goal. When the flood reaches the robot, you follow the numbers downhill and that is your path. **A*** is the same algorithm with one addition: a **heuristic** that guesses how far each cell still is from the target, so the flood is steered towards the goal instead of spreading in all directions. Same path, fewer cells examined.

Sampling-based methods do not enumerate anything. **RRT** (Rapidly-exploring Random Tree) throws a random point into the space, finds the nearest point already on its tree, and grows a short branch towards it, discarding the branch if it hits an obstacle. Repeat until the tree reaches the goal. **RRT*** adds a rewiring step: each time a new node is added, nearby nodes check whether routing through the newcomer would be cheaper, and reconnect if so. Plain RRT finds *a* path; RRT* converges towards the *best* one.

What this project uses

From `config/nav2_params.yaml`:

<code>plugin</code>	<code>nav2_navfn_planner/NavfnPlanner</code>
<code>use_astar</code>	<code>false</code>
<code>allow_unknown</code>	<code>true</code>
<code>tolerance</code>	<code>0.5</code>

`use_astar: false` means it is **Dijkstra**, not **A***. Flipping that one word to `true` gives you **A***; on a building 277×232 cells across the difference is a few milliseconds, which is why it was never worth changing.

No planner was written for this project. `NavFn` is a stock Nav2 plugin. It was configured, not implemented — and the same is true of the DWB controller.

`allow_unknown: true` — and why it compounds a defect

This setting permits the planner to route through cells the map records as unknown. Read §6.3 on how unknown cells are encoded before deciding whether that is safe here; the short version is that this project had *two* independent mechanisms letting paths escape into unmapped space, and only one of them was a map defect.

So why no RRT? Because it would be strictly worse here. RRT exists for problems where the space is too large or too awkward to enumerate — a seven-jointed robot arm, or a car that cannot turn on the spot manoeuvring in a tight yard. This robot drives on a flat 2D grid of about 64,000 cells. Enumerating them is cheap; Dijkstra is guaranteed to find the shortest path on that grid if one exists, and to report failure if none does. RRT guarantees neither: it is only *probabilistically* complete (it will find a path eventually, with probability approaching one), and basic RRT’s path is not shortest — which is exactly why RRT* had to be invented. You would be trading two guarantees for nothing. Nav2 ships no RRT planner for this reason.

6.1.3 Three places this system samples, for three different reasons

The word “sampling” appears in three unrelated places in this stack, and conflating them is a common source of confusion.

Component	What is sampled	How many
AMCL	poses (x, y, θ) — particles	500–2000, KLD-adaptive
DWB controller	velocity pairs (v_x, v_θ)	$20 \times 20 = 400$ per cycle
NavFn planner	<i>nothing</i> — exhaustive search	every reachable cell

So the global planner is search-based, the local controller is sampling-based — but it samples *control commands*, not positions, which is what makes it a different thing from RRT — and the localiser is sampling-based over states. Three uses of one word, three different objects being sampled.

6.2 Where AMCL is in that picture — and why it matters

Look again at Figure 6.1. **AMCL is not a box.**

The robot’s pose enters from the left as **TF Transforms** — an ambient input that every server reads. There is no component in the diagram whose job it is to ask whether that pose is right. The planner takes it as the start of the path. The controller takes it as the robot’s current position when deciding whether the goal has been reached. The behaviour tree takes it as the input to the “goal reached” condition.

The structural cause of the initial failure

Nav2’s layering is excellent engineering: the navigation stack does not need to know how localisation works, and localisation can be swapped out without touching navigation. But the same layering means **there is nowhere in the architecture for the question “is the pose correct?” to be asked**. The seam between AMCL and Nav2 is a transform, and a transform carries no confidence.

When AMCL believed the robot was at the dining table and the robot was 15.6m away, every component downstream behaved perfectly — against a false premise. Nav2 reported “succeeded” because, in the only frame it can see, the robot had arrived.

This is why the evaluation in Part 7 does not score arrival using `/amcl_pose`. Using AMCL to check AMCL would be measuring the belief against itself.

6.3 Costmaps, footprint and inflation

A **costmap** is a grid over the building where each cell holds a number saying how bad it is to be there.

Costmap values are 0–100, not 0–255

Nav2 publishes costmaps as `nav_msgs/OccupancyGrid`, whose values run 0 to 100 (-1 = unknown). 0 = free, 99 = inscribed/lethal. The internal C++ representation uses 0–254 and much documentation quotes those numbers, which is a genuine trap: a threshold of 253 applied to a published grid marks nothing at all. This project hit exactly that error during diagnosis.

Three layers stack up:

- **static layer** — the map produced by SLAM;
- **obstacle layer** — what the laser sees right now;
- **inflation layer** — a soft halo painted around every obstacle, so the planner prefers the middle of a corridor to hugging a wall.

“Unknown” is read as “free”, and here is the arithmetic

The static layer comes from a `.pgm` image, a grey-scale picture in which each pixel is one map cell. `map_saver` writes an *unknown* cell as the grey value 205. `map_server` then converts a grey value g into an occupancy probability:

$$\text{occ} = \frac{255 - g}{255} = \frac{255 - 205}{255} = \frac{50}{255} = 0.196.$$

The map’s own metadata file declares `free_thresh: 0.25`. Since $0.196 < 0.25$, the cell is classified **free**.

Read that again, because it is not what the word “unknown” suggests. A cell the mapping process never observed is handed to the planner as open floor. In this project the first map had 1066 such cells sitting on the outer boundary of the building, and the planner cheerfully routed paths through them and out of the house. Sealing that perimeter was one of the corrections. Combined with `allow_unknown: true` (§6.1.2), unmapped space was being admitted twice over.

Inflation cost decays exponentially with distance d from the nearest obstacle:

$$\text{cost}(d) = (\text{lethal} - 1) \exp(-\text{cost_scaling_factor} \times (d - r_{\text{inscribed}})), \quad d \leq \text{inflation_radius}. \quad (6.1)$$

This project uses `cost_scaling_factor: 6.0` and `inflation_radius: 0.55`.

6.3.1 Footprint, not radius

ROMR is not round. Describing it by a single `robot_radius` forces you to use the half-diagonal, which over-states the robot’s width and blocks planning through this building’s doorways. This project therefore supplies an explicit polygon:

```
footprint: "[[0.292, 0.304], [0.292, -0.304], [-0.229, -0.304], [-0.229, 0.304]]"
```

That is a rectangle 0.521 m long by 0.608 m wide, with the origin offset forwards — 0.292 m of body ahead of the wheel axle and 0.229 m behind. The **inscribed radius** is the largest circle centred on the robot’s origin that fits entirely inside the polygon, so it is the distance to the nearest edge:

$$r_{\text{inscribed}} = \min\{0.292, 0.229, 0.304, 0.304\} = 0.229 \text{ m}.$$

That is the figure this project’s clearance analysis uses, and the one that appears in equation (6.1).

Debugging “failed to create plan”

When the planner refuses a goal, the cause is almost always one of three things, in this order of likelihood: (1) the *start* pose is inside an inflated cell, because AMCL has drifted the robot into a wall; (2) the *goal* pose is inside an inflated cell; (3) there is genuinely no corridor wide enough for the footprint. Check (1) first — in this project it was nearly always (1), and the cascade is described in Part 7 §7.5.

6.4 The goal checker and the progress checker

Two small plugins inside the Controller Server decide when a navigation is over. Their configuration in this project:

Plugin	Parameter	Value
SimpleGoalChecker	xy_goal_tolerance	0.25 m
	yaw_goal_tolerance	0.25 rad
	stateful	true
SimpleProgressChecker	required_movement_radius	0.3 m
	movement_time_allowance	10 s

The **goal checker** declares success when the robot is within 0.25 m of the goal *and* within 0.25 rad (14.3°) of the goal heading. `stateful: true` means that once the position test passes it stays passed while the robot rotates to the final heading.

The **progress checker** aborts when the robot fails to move 0.3 m within 10 s.

This is the cause of the ten unconfirmed arrivals

`SimpleProgressChecker` measures **translation only**. It computes the straight-line distance the robot’s position has moved and compares it to `required_movement_radius`. Rotation does not count.

So consider a robot that has arrived — position tolerance satisfied — and is now turning on the spot to meet the 0.25 rad heading tolerance. Its *position* is not changing. The progress checker sees no translation, waits 10 s, and aborts a navigation that had in fact succeeded.

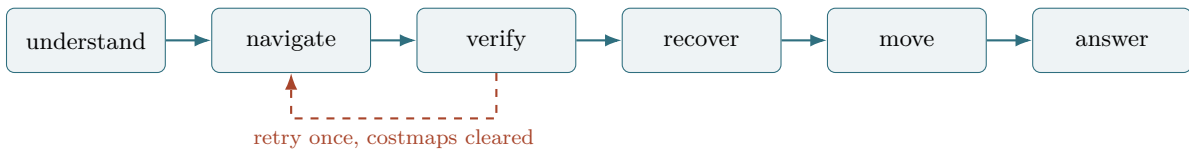
This was instrumented directly. At `waiting_area` the robot was recorded holding between 0.09 m and 0.24 m of the 0.25 m tolerance for 46 s, with commanded linear velocity flat at 0.000 m/s and commanded angular velocity consistently of one sign — turning, steadily, in one direction, getting nowhere. Ten trials in the corrected run ended this way: they arrived, and were reported as timeouts. Part 11 lists the fix.

6.5 What sits on top: the language layer

Nav2 takes a pose. A person says “go and wait in the dining room”. Something has to bridge that, and in this project it is a **LangGraph** state machine in the FastAPI backend.

The goal-epoch mechanism, and why it exists

The **verify** node waits for the `NavigateToPose` result. Early in the project it simply read the most recent outcome, and that is a bug: if a goal is superseded, the old goal’s result can still arrive and be attributed to the new one. Each dispatch therefore now opens a numbered epoch, and a result is only accepted if its epoch matches:



understand — an LLM (gpt-4.1-mini) parses the utterance into an intent and a place name
navigate — the place is looked up in the registry and dispatched to Nav2 as `NavigateToPose`
verify — wait for the real action result, tagged to this goal's epoch
recover — on failure, clear the costmaps and retry once
move / answer — any residual motion, then a natural-language reply

Figure 6.2: The LangGraph state machine that turns an utterance into a Nav2 goal and back into an answer. Every transition is traced to LangSmith, which is what makes the language layer auditable independently of the navigation layer.

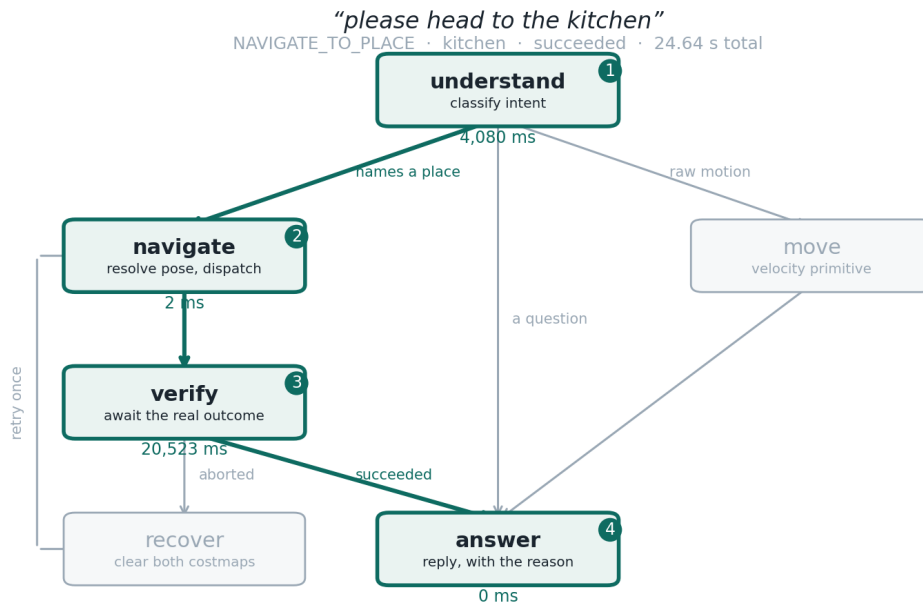


Figure 6.3: One command executing through the graph, as recorded. Regenerate with `scripts/plot_trace.py` over `backend/app/data/traces/graph_runs.jsonl`.

```
def _open_goal(self) -> int:
    self._goal_epoch += 1
    self.store.set("nav_outcome", None)
    self.store.set("is_navigating", True)
    return self._goal_epoch
```

This is in `backend/app/services/ros2_bridge.py` and it is one of the four faults tabulated in Part 11.

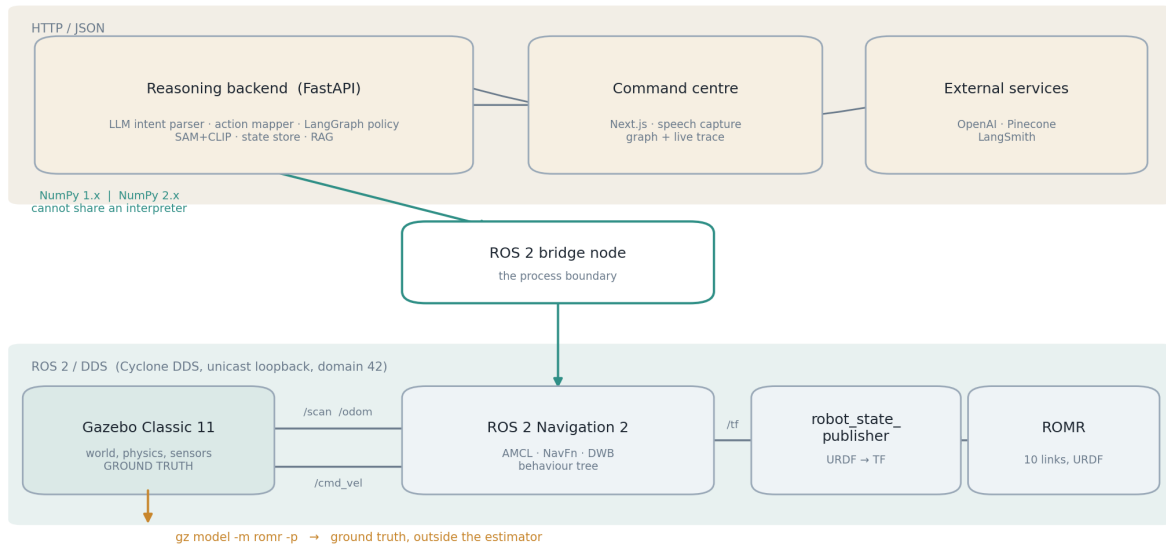


Figure 6.4: The complete system: language interface, reasoning backend, perception, and the ROS 2 / Gazebo simulation. AMCL and Nav2 occupy the right-hand portion; everything in this document concerns that portion and the measurements taken across it.

How the evaluation actually works

Everything so far has been machinery. This part explains what was measured, with what ruler, and why that ruler was chosen. It is written so that a secondary-school student can follow it end to end.

7.1 The one decision the whole evaluation rests on

When you ask a robot to go to the kitchen and it says “succeeded”, how do you check?

The tempting answer is: look at where the robot says it is, and see if that is near the kitchen. This is what almost every ROS tutorial does, and it is worthless, because it asks the robot to mark its own homework. In the initial run of this project it produced a 75% success rate for a system that actually arrived 19% of the time.

The evaluation therefore uses an **external ruler**: the simulator’s own record of where the robot’s body is. Gazebo knows this exactly, because Gazebo is the thing computing it. It is read with a single command:

```
gz model -m romr -p
```

which prints the model’s true pose. The validation script parses x , y and yaw from it.

Ground truth

Ground truth means: the real answer, obtained by a means independent of the system under test. Here it is available only because this is a simulation. On a physical robot it would require motion capture or a surveyed reference. That the ground truth exists at all is the single biggest advantage of doing this work in simulation, and it is why the false-positive rate could be measured rather than guessed at.

7.2 Three distances, and the triangle between them

For every trial the script records three numbers. Getting these three straight is most of the work of understanding the results.

Let

- $\mathbf{g} = (g_x, g_y)$ be the **goal** — the registry coordinates of the destination;
- $\mathbf{t} = (t_x, t_y)$ be the **truth** — where the robot’s body actually is, from Gazebo;
- $\mathbf{b} = (b_x, b_y)$ be the **belief** — where AMCL says the robot is.

Then, using the ordinary distance formula from school geometry $d = \sqrt{(\Delta x)^2 + (\Delta y)^2}$:

$$\begin{aligned}
 e_{\text{true}} &= \|\mathbf{t} - \mathbf{g}\| = \sqrt{(t_x - g_x)^2 + (t_y - g_y)^2} && \text{how far the robot really is from the goal} \\
 e_{\text{AMCL}} &= \|\mathbf{b} - \mathbf{g}\| = \sqrt{(b_x - g_x)^2 + (b_y - g_y)^2} && \text{how far the robot \textit{thinks} it is from the goal} \\
 e_{\text{drift}} &= \|\mathbf{b} - \mathbf{t}\| = \sqrt{(b_x - t_x)^2 + (b_y - t_y)^2} && \text{how wrong the robot’s self-knowledge is}
 \end{aligned} \tag{7.1}$$

7.5 Trial independence, and why re-seeding is itself a result

Each trial begins correctly localised. Before dispatching a command the script checks the gap between belief and truth, and if it exceeds 1.0m it re-seeds AMCL from ground truth. If the robot has left the mapped region entirely it is teleported back to the spawn pose. Both events are counted, and the count is reported.

Why this is honest rather than convenient

Without re-seeding, a single divergence poisons every subsequent trial. The mechanism is a cascade: the robot follows a valid map-frame path into a wall; its believed pose then lands inside an inflated costmap cell; and the planner thereafter refuses *every* goal, because it cannot plan from a start pose it considers occupied. That is a real property of the system — and it is reported as such, in Part 8 §8.3 — but averaged into a per-command success rate it would measure trial *ordering* rather than language understanding.

Separating the two, and reporting the re-localisation count as its own number, is the honest arrangement. In the corrected run the count is 0/61: no trial needed re-seeding, and the robot never left the map. That zero is itself one of the strongest results in the study.

7.6 What the RViz window shows

What you see	What it is
Grey/black occupancy grid	the static map, <code>custom_map_v2</code>
Speckle of small dots	the AMCL particle set; each dot is one (x, y, θ) hypothesis. Its <i>width</i> is σ made visible
White polygon	the published footprint, at the believed pose
Red/orange dots on walls	the current <code>/scan</code> , projected from the believed pose
Cyan/purple/pink wash	the inflation layer, equation (6.1); cyan is nearest to lethal
Thin line to the goal	the global plan from the Planner Server
2D Pose Estimate	publishes <code>/initialpose</code> ; this is how you re-seed AMCL by hand
Nav2 Goal	publishes a <code>NavigateToPose</code> goal directly, bypassing the language layer

The single most useful thing to look at

If the laser scan (the dots) lines up with the walls of the map, localisation is working. If the scan is rotated or offset from the walls, it is not — *regardless of what the pose estimate says*. The scan overlay is an independent visual check of exactly the quantity equation (5.4) scores numerically.

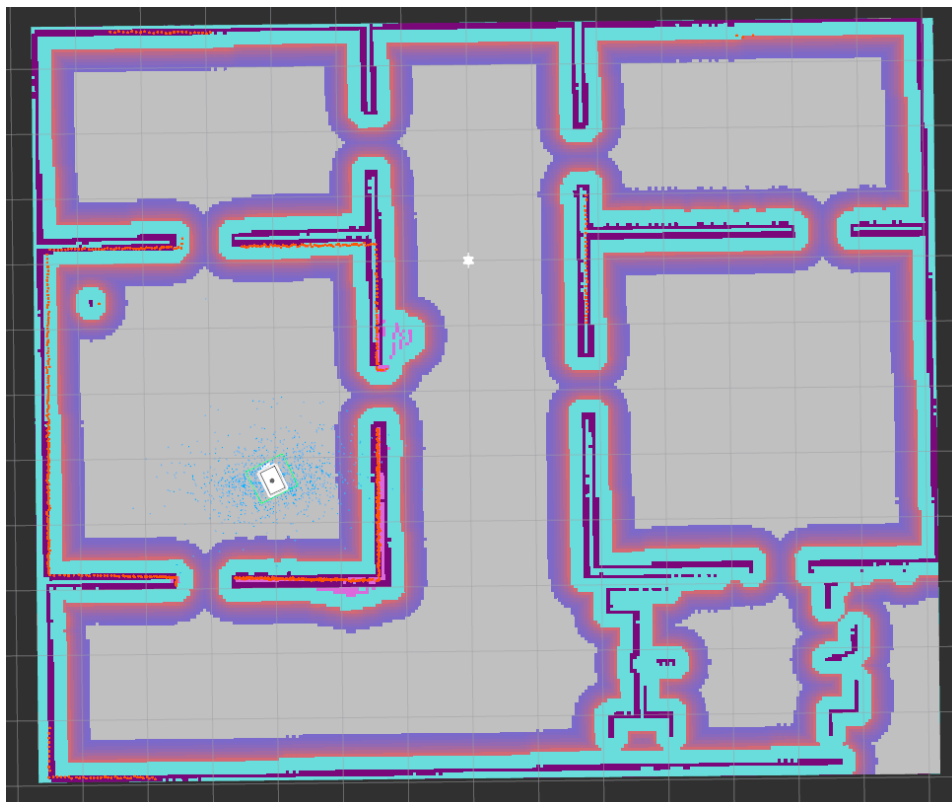


Figure 7.2: RViz during operation, viewed from directly above. Everything discussed in Parts 3 to 6 is visible in this one window. **Grey** is mapped free space and the dark outlines are walls, from the static layer. The **cyan-to-purple-to-pink band** hugging every wall is the inflation layer of equation (6.1) — cyan is nearly lethal, pink is the outer edge of the halo at `inflation_radius` = 0.55 m; the planner will not route the robot through the cyan and prefers to stay out of the purple. The **white rectangle** at mid-left is the robot's published footprint, the 0.521 m by 0.608 m polygon of §6.3, drawn at the believed pose. The **blue speckle** spread around it *is* the particle set $\mathcal{X}_t$ of equation (3.1) — every dot is one complete hypothesis about where the robot is. The **red and orange dots** along the walls are the current laser scan, projected from the believed pose.

Every number, worked by hand

This part reproduces the results tables from arithmetic you can do on paper. For each table, at least two entries are computed in full. Where a number cannot be obtained by hand — because it requires processing hundreds of laser beams, or running the filter — the exact procedure is given instead, with the command that produces it.

Two source files are used throughout:

Run	File	Rows
Initial (before correction)	<code>validation_results_groundtruth.csv</code>	54
Corrected (after correction)	<code>results/validation_2026-08-04_fixed.csv</code>	61

Both have the same eleven columns:

`n`, `phrasing`, `expected`, `resolved`, `correct`, `outcome`, `error_true_m`, `error_amcl_m`, `amcl_drift_m`, `reseeded`, `seconds`.

8.1 The distance formula, twice, on real coordinates

Every e and d in the tables comes from one formula, the Pythagorean distance:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$

Worked example 1: how far is the kitchen from home?

From the place registry (`backend/app/data/places/new_world_places.yaml`):

$$\text{home} = (-3.5378, -2.6976), \quad \text{kitchen} = (-5.0793, 1.9436).$$

Step 1 — the differences.

$$\Delta x = -5.0793 - (-3.5378) = -1.5415, \quad \Delta y = 1.9436 - (-2.6976) = 4.6412.$$

(Careful with the double negatives: subtracting a negative adds.)

Step 2 — square them.

$$(-1.5415)^2 = 2.376222, \quad (4.6412)^2 = 21.540737.$$

Step 3 — add.

$$2.376222 + 21.540737 = 23.916960.$$

Step 4 — square root.

$$d = \sqrt{23.916960} = 4.8905 \text{ m}.$$

Worked example 2: parlour to garage

$$\begin{aligned} \text{parlour} &= (-4.5742, -1.0034), & \text{garage} &= (0.3124, -6.4615). \\ \Delta x &= 0.3124 + 4.5742 = 4.8866, & \Delta y &= -6.4615 + 1.0034 = -5.4581. \\ 4.8866^2 &= 23.878860, & (-5.4581)^2 &= 29.790856. \end{aligned}$$

$$d = \sqrt{23.878860 + 29.790856} = \sqrt{53.669716} = 7.3260 \text{ m.}$$

Note that squaring removes the sign: it does not matter which point you call first.

These are distances between two *known* points. In the results, e_{true} is the same formula applied between the goal and the robot's Gazebo pose, and e_{AMCL} between the goal and AMCL's pose. The CSV stores the resulting distances rather than the raw poses, so the individual (x, y) values are not recoverable from it — which is why the next section uses a different, and arguably stronger, check.

8.2 Checking the recorded data against itself

Even without the raw poses, the three recorded distances must obey the **triangle inequality**: for any three points, no side of the triangle they form can be longer than the other two added together, nor shorter than their difference.

$$|e_{\text{true}} - e_{\text{AMCL}}| \leq d_{\text{drift}} \leq e_{\text{true}} + e_{\text{AMCL}}. \quad (8.1)$$

This is a real audit: if the columns had been recorded at inconsistent moments — if the robot had moved between reading its belief and reading its truth — equation (8.1) would fail.

Two trials checked

Trial 18, initial run, “take me to the dining room”. Recorded: $e_{\text{true}} = 15.631$, $e_{\text{AMCL}} = 0.241$, $d_{\text{drift}} = 15.432$.

$$|15.631 - 0.241| = 15.390, \quad 15.631 + 0.241 = 15.872.$$

Is $15.390 \leq 15.432 \leq 15.872$? Yes. ✓

Trial 43, initial run, “head to the dining area”. Recorded: 15.664, 0.208, 15.465.

$$|15.664 - 0.208| = 15.456, \quad 15.664 + 0.208 = 15.872.$$

Is $15.456 \leq 15.465 \leq 15.872$? Yes. ✓

What the near-equality tells you. In both cases d_{drift} is only just above the lower bound. A triangle whose third side almost equals the difference of the other two is almost *flat*: the three points are nearly in a straight line, with the belief sitting between the goal and the truth, right next to the goal.

Quantify it with the law of cosines. The angle at the belief vertex is

$$\cos \phi = \frac{e_{\text{AMCL}}^2 + d_{\text{drift}}^2 - e_{\text{true}}^2}{2 e_{\text{AMCL}} d_{\text{drift}}}.$$

For trial 43:

$$\cos \phi = \frac{0.208^2 + 15.465^2 - 15.664^2}{2 \times 0.208 \times 15.465} = \frac{0.043 + 239.167 - 245.361}{6.433} = \frac{-6.151}{6.433} = -0.9562,$$

so $\phi = 163.0^\circ$ — nearly a straight line.

The physical meaning: AMCL had planted its belief essentially *on the goal*, and the robot was fifteen and a half metres away along almost exactly the line from the goal through the belief. This is not a filter that is unsure. It is a filter that has confidently converged on the wrong place. Compare Part 3 §3.2.3 on the support condition.

8.3 Table — validity audit: 16 trials, not 54

The initial run has 54 rows, but only some of them represent a navigation that actually ran. The audit splits them by duration.

	≥ 10 s	< 10 s
Trials	16	38
Reported success	12	0
Arrived ($e_{\text{true}} \leq 0.5$ m)	3	0
Median e_{true} (m)	6.35	5.98
Median d_{drift} (m)	6.25	0.54

Entry 1: the split, 16 and 38

Read the `seconds` column of `validation_results_groundtruth.csv` and sort it. The values fall into two clearly separated groups:

Short group — 4.4, 4.4, 4.8, 4.9, ..., 5.8, 9.9 seconds. These are trials where the planner refused the goal immediately: the command was parsed, dispatched, and rejected in about five seconds. Counting them: **38**.

Long group — these sixteen, in seconds:

25.2, 26.0, 27.8, 63.7, 64.3, 67.6, 74.2, 76.1,
83.2, 86.8, 88.8, 94.7, 94.9, 95.4, 131.9, 143.4.

Counting them: **16**.

$38 + 16 = 54$. ✓ There is no trial between 9.9 s and 25.2 s, so the 10 s cut is unambiguous.

Entry 2: median e_{true} for the 16-trial group

The sixteen true errors, sorted (metres):

0.330, 0.426, 0.467, 1.693, 2.071, 5.127, 5.182, **5.202**,
7.491, 7.524, 8.733, 8.793, 11.264, 15.612, 15.631, 15.664.

16 is even, so the median is the mean of the 8th and 9th:

$$\frac{5.202 + 7.491}{2} = \frac{12.693}{2} = 6.3465 \rightarrow 6.35 \text{ m. } \checkmark$$

Entry 3: median d_{drift} for the same group

The sixteen drifts, sorted (metres):

0.394, 0.531, 0.576, 2.524, 2.677, 4.982, 5.035, **5.295**,
7.204, 7.221, 8.668, 8.702, 11.132, 15.204, 15.432, 15.465.

$$\frac{5.295 + 7.204}{2} = \frac{12.499}{2} = 6.2495 \rightarrow 6.25 \text{ m. } \checkmark$$

Why the audit matters

All 12 reported successes are in the 16-trial group; the 38 short trials reported nothing. So the false-positive rate is a statement about the 16, and reporting it over all 54 would dilute

it with trials that never navigated. This is a deliberate narrowing that makes the headline number *worse*, not better, and it is the right thing to do.

8.4 Table — reported versus actual arrival, initial run

Quantity	Count	Rate
Trials	16	—
Reported success by the stack	12	75%
Arrived within 0.5 m	2 of those 12	17%
Reported success that was false	10 of 12	83%

Entry 1: the 75%

Count rows in the 16-trial group whose `outcome` column reads `succeeded`. They are trials 18, 19, 20, 21, 22, 38, 39, 40, 41, 42, 43, 44 — twelve of them.

$$\frac{12}{16} = 0.75 = 75\%. \checkmark$$

Entry 2: the 17% and the 83%

Take those same twelve rows and read their e_{true} :

n	phrasing	e_{true}	≤ 0.5 m?
18	take me to the dining room	15.631	no
19	please move to the garage	8.793	no
20	go park in the garage	8.733	no
21	go to the storage area	5.127	no
22	navigate home	0.467	yes
38	please go to the store area	5.202	no
39	move to storage	5.182	no
40	I need you in the kitchen	7.491	no
41	please head to the kitchen	7.524	no
42	move to the living room	11.264	no
43	head to the dining area	15.664	no
44	head back home	0.426	yes

Two “yes” out of twelve:

$$\frac{2}{12} = 0.1667 = 16.7\% \approx 17\% \checkmark$$

and therefore

$$\frac{10}{12} = 0.8333 = 83.3\% \approx 83\% \text{ false. } \checkmark$$

Read the table above once more. Both of the genuine arrivals are *home*, which is only 0.96 m from the spawn pose — the robot barely had to move, so odometry barely had a chance to drift. Every trial that required a real journey failed. That single observation is the diagnosis, and it is visible from arithmetic alone.

8.5 Table — arrivals as a function of the threshold

Threshold (m)	0.3	0.5	0.6	1.0
Arrivals	0	3	3	4

The point of this table is to show the headline result does not depend on where the arrival line is drawn.

All four entries, by hand

Sort every e_{true} in the initial run and look at the small end:

0.330, 0.426, 0.467, 0.681, 1.693, 1.782, ...

Now count how many are $\leq$ each threshold:

- ≤ 0.3 : none. The smallest is 0.330. **0** ✓
- ≤ 0.5 : 0.330, 0.426, 0.467. **3** ✓
- ≤ 0.6 : the same three; 0.681 is above. **3** ✓
- ≤ 1.0 : those three, plus 0.681. **4** ✓

Even at double the chosen tolerance, four arrivals out of fifty-four.

A discrepancy worth fixing before publication

This particular table is computed over *all 54* rows, whereas the surrounding text works over the *16*-trial group. The two agree at 0.3, 0.5 and 0.6, but differ at 1.0: over the 16-trial group the count is 3, not 4, because the fourth arrival ($e_{\text{true}} = 0.681$, trial 17, “take the robot home”) is a 5.0s trial that never navigated. Either denominator is defensible; *stating which one is used* is not optional. The caption should read “over all 54 trials” or the 1.0 column should read 3.

8.6 Table — before and after correction

Measure	Initial	Corrected
Trials analysed	16	61
Intent resolved correctly	100%	60/61 (98%)
Reported success	12 (75%)	50 (82%)
Arrived within 0.5 m	3	60 (98%)
False positives among reported	10/12 (83%)	0/50
Median e_{true} (m)	6.35	0.231
Mean e_{true} (m)	—	0.259
Maximum e_{true} (m)	15.66	0.483
Median d_{drift} (m)	6.25	0.108
Mean d_{drift} (m)	6.94	0.189
Median trial duration (s)	5.1	57.2
Re-localisations required	—	0/61

Entry 1: median e_{true} of the corrected run = 0.231

Sixty of the 61 rows have an e_{true} (trial 49 resolved no place, so no error could be computed — see the next box). Sorted, the middle of the list reads:

..., 0.222, 0.224, 0.228, **0.230, 0.232**, 0.238, 0.244, 0.249, ...

With 60 values the median is the mean of the 30th and 31st:

$$\frac{0.230 + 0.232}{2} = \frac{0.462}{2} = 0.231 \text{ m. } \checkmark$$

Entry 2: mean e_{true} of the corrected run = 0.259

Sum all 60 values and divide by 60. The total is 15.551 m:

$$\bar{e} = \frac{15.551}{60} = 0.25918 \rightarrow 0.259 \text{ m. } \checkmark$$

Note mean > median (0.259 against 0.231), as always when a distribution has a longer tail on the high side. The gap is small here — 0.028 m — because the corrected run has no outliers. In the initial run the same gap was 0.60 m on a median of 6.35.

Entry 3: the 98% arrival rate

Count rows in the corrected run with $e_{\text{true}} \leq 0.5$. The maximum recorded value is 0.483 (trial 45), so *every* row that has a value at all passes. That is 60.

$$\frac{60}{61} = 0.98360 \approx 98\%. \checkmark$$

The one that does not is trial 49, “drive to the store”, where **resolved** is empty and **outcome** is **dispatched**. That is a *language* failure: the phrase “the store” did not resolve to **store_area**, so no goal was ever sent and no distance exists to measure. It is counted against the system — 60/61, not 60/60 — which is the conservative choice.

Entry 4: median d_{drift} = 0.108

Here all 61 rows have a value, including trial 49 (drift does not need a goal — it is belief against truth). 61 is odd, so the median is simply the 31st value of the sorted list, which is 0.108 m. $\checkmark$

Compare with the initial run’s 6.25 m: a factor of $6.25/0.108 = \mathbf{58}$ reduction in how badly the robot misunderstood its own position.

Entry 5: median duration 5.1 $\rightarrow$ 57.2 s

Initial run, all 54 durations sorted: the 27th and 28th are both 5.1 s, so the median is 5.1 s. $\checkmark$

Corrected run, 61 durations sorted: the 31st is 57.2 s. $\checkmark$

This row is not padding. A median of five seconds means the typical initial trial *never navigated at all* — it was refused by the planner. A median of fifty-seven seconds means the typical corrected trial drove across the building. The two runs are not comparing good navigation with bad navigation; they are comparing navigation with its absence.

8.7 The pattern nobody had looked for: error against distance

Every table so far reports a single number for the whole run. That hides something, and what it hides turns out to be the clearest signal in the entire dataset.

Take the 16 valid trials of the initial run, group them by which room was asked for, and sort the groups by how far that room is from where the robot starts, at $(-2.92, -3.43)$.

The two distances, by hand

Furthest — the dining room, at $(5.6325, 1.5407)$:

$$\begin{aligned}\Delta x &= 5.6325 - (-2.92) = 8.5525, & \Delta y &= 1.5407 - (-3.43) = 4.9707 \\ \Delta x^2 &= 73.1453, & \Delta y^2 &= 24.7079\end{aligned}$$

$$d = \sqrt{73.1453 + 24.7079} = \sqrt{97.8532} = 9.89 \text{ m.}$$

Nearest — home, at $(-3.5378, -2.6976)$:

$$\begin{aligned}\Delta x &= -3.5378 - (-2.92) = -0.6178, & \Delta y &= -2.6976 - (-3.43) = 0.7324 \\ \Delta x^2 &= 0.3817, & \Delta y^2 &= 0.5364\end{aligned}$$

$$d = \sqrt{0.3817 + 0.5364} = \sqrt{0.9181} = 0.96 \text{ m.}$$

Destination	Distance (m)	Trials	Median error (m)	Range (m)	Ranks d / e
home	0.96	3	0.43	0.33–0.47	1 / 1
parlour	2.94	2	6.67	2.07–11.26	2 / 3
store_area	2.95	4	5.15	1.69–5.20	3 / 2
garage	4.43	2	8.76	8.73–8.79	4 / 5
kitchen	5.79	2	7.51	7.49–7.52	5 / 4
dining_room	9.89	3	15.63	15.61–15.66	6 / 6

Read the last column downwards. The order of the rooms by distance and the order of the rooms by error are *almost the same list*. Only two adjacent pairs are swapped.

8.7.1 Putting a number on “almost the same list”

A **correlation coefficient** is one number saying how closely two quantities move together. It runs from -1 (perfect opposition) through 0 (no relationship) to $+1$ (perfect agreement).

The easiest one to compute by hand is **Spearman’s rank correlation** ρ , which ignores the actual values and looks only at the *ordering*. That is exactly what we want here. Its formula, when no two ranks tie, is

$$\rho = 1 - \frac{6 \sum_i d_i^2}{n(n^2 - 1)}, \quad (8.2)$$

where d_i is the difference between the two ranks of item i , and n is how many items there are.

Spearman’s ρ over the six destinations

Take the rank differences straight from the table’s last column:

Destination	rank by d	rank by e	d_i	d_i^2
home	1	1	0	0
parlour	2	3	-1	1
store_area	3	2	+1	1
garage	4	5	-1	1
kitchen	5	4	+1	1
dining_room	6	6	0	0
			$\sum d_i^2$	4

With $n = 6$:

$$\rho = 1 - \frac{6 \times 4}{6 \times (36 - 1)} = 1 - \frac{24}{210} = 1 - 0.1143 = \mathbf{0.886}. \checkmark$$

Scored over the 16 individual trials rather than the six room averages, the same coefficient is 0.821 and the ordinary linear correlation is 0.911. Those are the two figures quoted in the thesis.

What a correlation does *not* tell you

A high ρ says two orderings agree. It does not say one causes the other, and it does not license a significance test here — there are six rooms, they were not randomly chosen, and the trials within a room are not independent of one another. The coefficient is quoted as a compact description of the table above it, nothing more. The table is the evidence; the number is shorthand.

8.7.2 Why this matters more than the coefficient does

Forget the statistics for a moment and look at the raw trials.

- All three `dining_room` trials — the furthest room, at 9.89 m — finished 15.61, 15.63 and 15.66 m from the goal. That is a spread of 0.05 m across three independent runs.
- All three `home` trials — the nearest room, at 0.96 m — finished 0.33, 0.43 and 0.47 m from the goal.

Same map. Same parameter file. Same afternoon. The only thing that differed was how far the robot had to travel.

This distinguishes two kinds of fault

A fault *in the map* belongs to a place. A wall drawn in the wrong spot ruins one room and leaves the others alone — and it does not care how far away that room is.

A fault *in the model* — in the odometry noise settings, or in the laser range assumption — belongs to the *path*. Its effect accumulates with every metre driven. Twice the distance, roughly twice the error.

The table shows error tracking distance almost perfectly. That is the signature of the second kind, and it is why §8.10 goes on to measure the sensor and motion models rather than re-examining the map. The stability of the dining-room trials points the same way, for the reason given in §2.1: a filter that has settled on a wrong answer returns to that same wrong answer every time.

8.7.3 And after the corrections

Run the same analysis on the corrected data and the pattern is gone. The linear correlation falls from 0.911 to 0.147; the rank correlation from 0.821 to 0.121. The eight `dining_room` trials — still the furthest room — finish between 0.16 and 0.42 m from the goal, which is the same band as every other room in the house.

Be honest about why that number is small

The correlation collapsed partly because there is nothing left for distance to explain: *no* trial in the corrected run missed by more than 0.483 m, at any range. When every value is squeezed into a band a few centimetres wide, a correlation coefficient has almost nothing to bite on.

So the defensible claim is “**large errors at long range have disappeared**”, not “distance and error are now proven independent”. The first is what the data shows. The second would need failures deliberately induced across a range of distances, which is a study that has not been run.

8.8 Table — the confusion matrix, precision and recall

This is the table that carries the argument, so it is worked in full.

Treat the stack’s “succeeded” report as a *decision* about whether the robot arrived, and score that decision against ground truth. Four outcomes are possible.

		ACTUALLY ARRIVED		
		yes	no	
STACK REPORTED	success	TP true positive said yes, was there	FP false positive said yes, was not	$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$ <i>when it says yes, how often is it right?</i>
	failure	FN false negative said no, was there	TN true negative said no, was not	$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$ <i>of the times it was there, how often did it say so?</i>

Figure 8.1: The four possible outcomes of treating a success report as a decision, and the two rates computed from them. Precision is a property of the *claims*; recall is a property of the *opportunities*.

	Initial ($n = 16$)	Corrected ($n = 61$)
True positives	2	50
False positives	10	0
False negatives	1	10
True negatives	3	1
Precision	0.167	1.000
Recall	0.667	0.833

The initial run, all six numbers

Step 1 — classify each of the 16 trials. Reported success: 12 trials. Arrived ($e_{\text{true}} \leq 0.5$): 3 trials, namely $n = 1$ (0.330), $n = 22$ (0.467), $n = 44$ (0.426).

Step 2 — cross-tabulate.

- TP = reported *and* arrived. Of the three arrivals, 22 and 44 were reported; $n = 1$ was aborted. So **TP** = 2.
- FP = reported *but did not* arrive. $12 - 2 = 10$.
- FN = arrived *but not* reported. $3 - 2 = 1$ (that is $n = 1$, “go home”).
- TN = neither. $16 - 2 - 10 - 1 = 3$.

Check: $2 + 10 + 1 + 3 = 16$. ✓

Step 3 — precision.

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{2}{2 + 10} = \frac{2}{12} = 0.1667 \rightarrow \mathbf{0.167} \checkmark$$

Step 4 — recall.

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{2}{2 + 1} = \frac{2}{3} = 0.6667 \rightarrow \mathbf{0.667} \checkmark$$

Step 5 — read it. Precision 0.167 says: *five times out of six, “succeeded” was a lie*. Recall 0.667 says: *when the robot did get there, it noticed two times out of three*. Notice that recall is not terrible while precision is catastrophic — the system’s problem was not that it missed arrivals, it was that it invented them.

The corrected run, all six numbers

Step 1. Reported success: 50. Arrived: 60.

Step 2. Every one of the 50 reported successes has $e_{\text{true}} \leq 0.5$ — the largest is 0.483 — so:

- **TP** = 50
- **FP** = $50 - 50 = 0$
- **FN** = $60 - 50 = 10$
- **TN** = $61 - 50 - 0 - 10 = 1$ (trial 49, which never navigated)

Check: $50 + 0 + 10 + 1 = 61$. ✓

Step 3 — precision.

$$\frac{50}{50 + 0} = \frac{50}{50} = \mathbf{1.000} \checkmark$$

Step 4 — recall.

$$\frac{50}{50 + 10} = \frac{50}{60} = 0.8333 \rightarrow \mathbf{0.833} \checkmark$$

Step 5 — read it. Precision 1.000 says: *every claim of success was true*. Recall 0.833 says: *one arrival in six went unreported*. The error has changed *kind*. The system no longer lies; it is merely modest. For a robot that is the right direction to fail in — an unreported arrival is a nuisance, a reported non-arrival is a hazard — but it is still a defect, and §8.9 identifies it exactly.

Precision and recall pull against each other

You can always get precision 1.000 by never claiming success, and recall 1.000 by always claiming it. Reporting only one of them is therefore meaningless. This project reports both, and the honest summary is: precision went $0.167 \rightarrow 1.000$ (a real and large improvement) while recall went $0.667 \rightarrow 0.833$ (also an improvement, but a smaller one, and there is a known reason it is not 1.000).

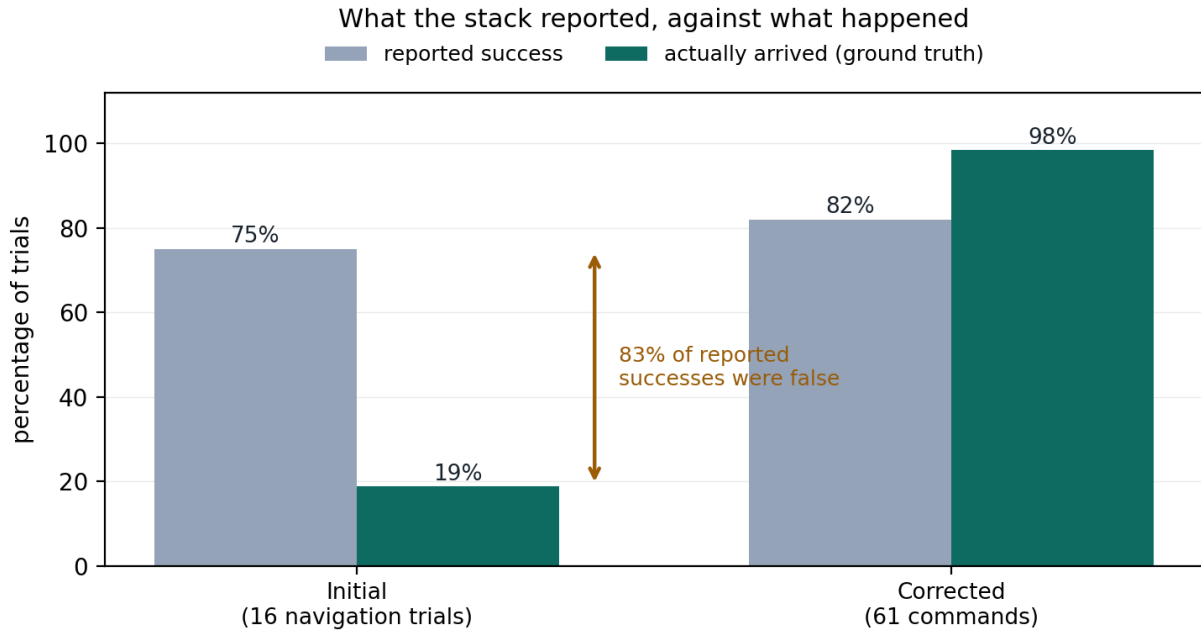


Figure 8.2: The two runs on the same axes. Regenerate with `scripts/plot_validation.py`.

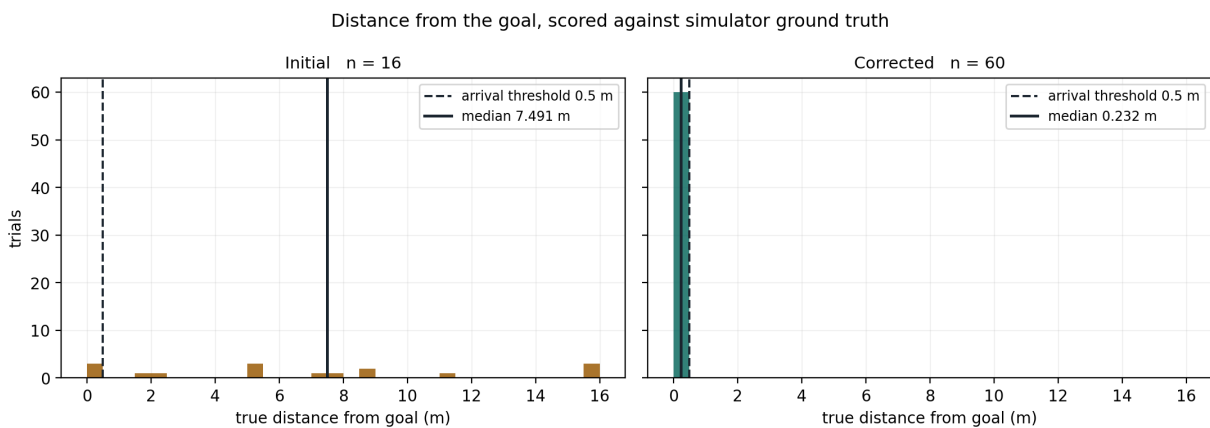


Figure 8.3: Left: each trial placed at (error the robot believed it had, error actually measured). Points on the diagonal are honest beliefs; points high above it are the false positives. Right: how far the estimate strayed, sorted. Regenerate with `scripts/plot_validation.py`.

8.9 Table — the ten unconfirmed arrivals

Destination	Count	e_{true} range (m)	Duration (s)
waiting_area	4	0.12–0.23	≈ 185
home	3	0.22–0.30	≈ 185
store_area	2	0.25–0.39	140–185
garage	1	0.25	167

The whole table, by hand

Select rows of the corrected run where `outcome` is *not succeeded* but $e_{\text{true}} \leq 0.5$. There are exactly ten:

n	destination	outcome	e_{true}	seconds
4	home	timeout	0.287	184.8
8	waiting_area	timeout	0.190	185.0
10	waiting_area	timeout	0.143	185.1
11	garage	timeout	0.252	167.4
15	home	timeout	0.222	184.9
17	waiting_area	timeout	0.230	185.4
50	home	timeout	0.298	184.8
51	store_area	timeout	0.389	139.8
54	store_area	timeout	0.249	184.8
61	waiting_area	timeout	0.117	185.2

Tally by destination: `waiting_area` appears in rows 8, 10, 17, 61 — **4**. `home` in rows 4, 15, 50 — **3**. `store_area` in rows 51, 54 — **2**. `garage` in row 11 — **1**. Total $4 + 3 + 2 + 1 = 10$. ✓
 Ranges: for `waiting_area`, the smallest e_{true} is 0.117 and the largest is 0.230, giving “0.12–0.23”. ✓ For `home`: 0.222 to 0.298, giving “0.22–0.30”. ✓

Three things fall out of this table immediately.

1. *Every one is a timeout, not an aborted.* The navigation did not fail; it never finished.
2. *Eight of the ten sat at almost exactly 185 s*, the script’s own timeout. They were still going when the clock ran out.
3. *Every one of them was already there*, with errors between 0.117 m and 0.389 m — comfortably inside the 0.5 m arrival threshold, and mostly inside Nav2’s own 0.25 m position tolerance.

A robot that is at the goal, is not moving, and never finishes, is a robot stuck on the *heading* half of the goal check — exactly the translation-only progress-checker fault of Part 6 §6.4.

8.10 Numbers that cannot be done by hand — and how they are obtained

Three tables in the results require running something. Here is exactly what, so the procedure is reproducible even though the arithmetic is not.

8.10.1 Scan–map agreement

Mean distance to nearest wall	0.047 m
Median distance	0.050 m
Within 0.10 m	66.8%
Within 0.20 m	100%

Procedure

1. Read the true pose (t_x, t_y, t_θ) from `gz model -m romr -p`.
2. Capture one message on `/scan`: 720 ranges z_k with angles θ_k .
3. For each beam, compute the point it hit, in map coordinates:

$$p_k = (t_x + z_k \cos(t_\theta + \theta_k), \quad t_y + z_k \sin(t_\theta + \theta_k)).$$

4. Build a **distance transform** of the occupancy grid — an image in which every free cell holds its distance to the nearest occupied cell. This is one call to `scipy.ndimage.distance_transform_edt` on the inverted map, multiplied by the map resolution in metres per cell.
5. Look up each p_k in that image. The result is 720 distances.
6. Report their mean, median, and the fraction below each threshold.

The rotation study repeats steps 3–6 with t_θ replaced by $t_\theta + \Delta$ for $\Delta \in \{45^\circ, 90^\circ, 135^\circ, 180^\circ\}$.

Why you cannot do it by hand: it is 720 trigonometric evaluations and 720 image lookups per pose, times five poses. The arithmetic per beam is trivial; the volume is not.

8.10.2 Localisation step response and the α sweep

Procedure

1. Bring up the simulator and Nav2 (Part 10).
2. Seed AMCL from ground truth: `scripts/seed.sh`.
3. Read the baseline: true pose from `gz`, believed pose and covariance from `/amcl_pose`.
4. Apply *one* isolated motion — stand still 8 s, or turn 90° , or drive 0.8 m — using `scripts/drive.py`.
5. Read the three quantities again:

$$e = \|\mathbf{b} - \mathbf{t}\|, \quad \Delta\theta = b_\theta - t_\theta, \quad \sigma_\theta = \sqrt{\Sigma_{\theta\theta}}$$

where $\Sigma_{\theta\theta}$ is the yaw variance from the covariance matrix, equation (5.6).

6. For the α sweep: edit `alpha1` and `alpha2` in `config/nav2_params.yaml`, restart Nav2, and repeat steps 2–5 for each value $\{0.25, 0.10, 0.05, 0.02, 0.01\}$. **Re-seed before every trial**, or you are measuring accumulated history rather than the parameter.

Why you cannot do it by hand: σ_θ is the spread over up to 2000 particles, each of which was moved by an independent random draw. There is no closed form; it has to be run.

8.10.3 Goal reachability and costmap occupancy

The claim that all eight destinations lie in one traversable region, with a minimum clearance of 1.09 m, and that 25.3% of the costmap is at or above the blocked value, comes from:

Procedure

1. Take the occupancy grid, threshold it into free / not-free.
2. Compute the distance transform (as above). Threshold it at the inscribed radius 0.229 m: cells with more clearance than that can hold the robot.
3. Run a **connected-component** labelling over the result. Check that all eight goal cells and the spawn cell carry the same label — if they do, they are mutually reachable.
4. The minimum clearance is the smallest distance-transform value among the eight goal cells: 1.09 m.
5. Subscribe to `/global_costmap/costmap` and count cells at or above the blocked value. **Remember the 0–100 scale** (Part 6 §6.3).

Where everything lives

All paths are relative to the repository root, `~/Multimodal-Robot-Autonomy`.

9.1 Result files

Path	Contents
<code>validation_results_groundtruth.csv</code>	The initial run. 54 rows. The “before” column of every comparison.
<code>results/validation_2026-08-04_fixed.csv</code> <code>results/validation_2026-08-04.csv</code>	The corrected run. 61 rows. The “after” column. An intermediate run: localisation fixed, navigation outcome identity not yet. 57 aborts, 3 reported successes, all 3 genuine. Useful for showing the two faults were independent.
<code>validation_results.csv</code>	An early <code>-mode chat</code> run without ground-truth columns. Superseded.
<code>results/figures/reported_vs_actual.png</code>	Before/after on shared axes
<code>results/figures/error_distribution.png</code>	Belief vs. truth scatter, and drift
<code>results/figures/alpha_sweep.png</code>	The α_1 sweep
<code>results/figures/graph_trace.png</code>	One LangGraph run, traced
<code>results/particles_*.png</code>	Particle clouds, tuned and mis-tuned
<code>results/particle_approximation.png</code>	Density vs. particle representation
<code>results/urdf/</code>	Kinematic tree renderings of the ROMR description
<code>backend/app/data/traces/graph_runs.jsonl</code>	One JSON object per LangGraph run: nodes visited, timings, outcome

A gotcha that cost real time

Trace persistence writes to `backend/app/data/traces/`, resolved in code as `Path(__file__).parents[1] / "data" / "traces"`. It was briefly `parents[2]`, which put the file in `backend/data/traces/` instead. Because the write is best-effort and wrapped in a `try`, nothing ever reported an error — the traces simply were not where anything looked for them. If a trace file seems empty, check both directories before assuming the run failed.

9.2 Configuration

Path	Contents
config/nav2_params.yaml	The central file of this study. AMCL, planner, controller, costmaps, recoveries. Heavily commented with the measurement behind each value. Pins DDS discovery to unicast loopback. Not optional on this host — see Part 10 §10.1. The eight destinations and their aliases. Goal coordinates come from here. API keys and model selection (gpt-4.1-mini). Never committed.
config/cyclonedds.xml	
backend/app/data/places/new_world_places.yaml	
backend/.env	
\$WS/src/romr_description/urdf/	The robot description
\$WS/src/multimodal_robot_autonomy/gazebo/	Gazebo plugins: differential drive, laser, camera, IMU. Here \$WS is ~/turtlebot3_ws

9.3 Scripts

Script	Purpose
scripts/env.sh	Shared environment. <i>Source</i> it, do not run it.
scripts/build.sh	Rebuild into ~/turtlebot3_ws — the workspace that is actually sourced
scripts/clean.sh	Kill every leftover process. Run before every session
scripts/sim.sh	Terminal 1: Gazebo, world, robot
scripts/nav2_rviz.sh	Terminal 2: Nav2 with RViz
scripts/backend.sh	Terminal 3: FastAPI on :8000
scripts/frontend.sh	Terminal 4: Next.js on :3000
scripts/seed.sh	Seed AMCL from Gazebo ground truth, then rotate to converge
scripts/drive.sh	Arrow-key teleoperation
scripts/record_place.py	Record the current pose as a named destination
scripts/watch.sh	Live readout of truth, belief, drift and σ
scripts/validate_commands.py	The evaluation suite. Produces the CSVs Belief-vs-truth scatter and drift plot The α sweep and before/after figures LangGraph trace figure Particle-cloud figures
scripts/plot_validation.py	
scripts/plot_results.py	
scripts/plot_trace.py	
scripts/particle_figure.py	
scripts/map_world.sh,	SLAM and map capture
scripts/save_map.sh	
scripts/record_screen.sh	Screen capture for the demonstration

9.4 Backend code that matters to this document

Path	Role
backend/app/services/ros2_bridge.py	The ROS 2 side: subscribes <code>/amcl_pose</code> (transient-local), <code>/scan</code> , the camera; dispatches <code>NavigateToPose</code> ; holds the goal-epoch mechanism
backend/app/services/command_graph.py	The LangGraph state machine of Figure 6.2
backend/app/services/intent_parser.py	Utterance $\rightarrow$ intent + place
backend/app/services/vector_store.py	<code>VectorStore</code> protocol with Chroma and Pinecone backends. Chroma returns a <i>distance</i> , Pinecone a <i>similarity</i> ; the adapter converts with <code>score = 1.0 - distance</code> so ranking is not silently inverted
backend/app/core/security.py	Rate limiting; <code>is_local()</code> exempts loopback

Running it: from a cold machine to a results CSV

10.1 Two settings that are not optional on this host

Before anything else, understand why `scripts/env.sh` exists. Two environment settings must be right or the stack will appear to work while delivering nothing.

The DDS problem

ROS 2 nodes find each other by multicast discovery. **This host has no multicast route**, and the default domain (0) collides with another machine on the network. The symptoms are maddening: `ros2 topic list` shows the topics, `ros2 topic echo` shows nothing, and every node reports itself healthy.

Two settings fix it, both applied by `env.sh`:

```
export CYCLONEDDS_URI="file://$REPO/config/cyclonedds.xml"
export ROS_DOMAIN_ID=42
```

The XML pins discovery to unicast loopback. The domain ID keeps this stack off the contended default.

`env.sh` also unsets the `SNAP_*`, `GTK_PATH` and `LD_LIBRARY_PATH` variables that VS Code's snap injects, which otherwise kill `rviz2` and `gz` with undefined symbol: `__libc_pthread_init`; and it detects the actual X display rather than assuming `:0`, because this machine runs on `:1`.

10.2 Start from a clean slate — always

Terminal 0

```
cd ~/Multimodal-Robot-Autonomy
scripts/clean.sh          # simulator, nav2, rviz
scripts/clean.sh --all    # ... and the backend (:8000) and frontend (:3000)
```

It prints `clean: nothing left running` when the slate is genuinely clear.

The single most common failure in this project is not a bug

It is a stale process. Three specific ones, each of which produces symptoms that look exactly like an application fault:

1. **A forgotten `teleop_twist_keyboard`.** It republishes its last twist in a loop, so a session left open after one keypress drives the robot forever. Measured: 588 mm of creep in 60 s with nobody touching the keyboard. This reads exactly like a physics bug and is not one.
2. **Extra `robot_state_publisher` processes.** Each publishes its own `/robot_description`

and TF, so a stale robot model keeps being spawned after the URDF has changed. Twenty-two of them accumulated here, because `pkill -x a b c` does *not* kill `a`, `b` and `c` — it takes one pattern and rejects the rest. `clean.sh` calls `pkill` once per process name for exactly this reason.

3. **slam_toolbox outliving its launch file.** It keeps publishing `map→odom`, silently fighting AMCL for the same transform the next time Nav2 comes up. Two nodes publishing the same transform is undefined behaviour, and the resulting pose jitters between them.

Note also the bracket idiom throughout `clean.sh`: `pkill -f "[g]azebo_romr.launch.py"`. The brackets stop the pattern from matching `pkill`'s own command line, which would make the script kill the shell that launched it.

10.3 The five terminals

Bring-up, in order

```
# Terminal 1 - simulator, world and robot
scripts/sim.sh

# Terminal 2 - navigation with RViz, so the initial pose can be set by hand
scripts/nav2_rviz.sh

# Terminal 3 - reasoning backend, FastAPI on :8000
scripts/backend.sh

# Terminal 4 - command centre, Next.js on :3000
scripts/frontend.sh

# Terminal 5 - working shell for seeding, driving and validation
source scripts/env.sh
```

Wait for each to settle before starting the next. Terminal 2 in particular should be given time to report all its lifecycle nodes active.

If a newly added file seems to be ignored

Rebuild — but with the right script:

```
scripts/build.sh
```

The package *source* lives in this repository, but the colcon workspace is `~/turtlebot3_ws`, which carries this directory as a symlink under `src/` and is what `env.sh` sources. Running `colcon build` from the repository root succeeds, prints no warning, and creates a second install tree that nothing ever reads.

Existing files survive that mistake, because `-symlink-install` links them back to *source*. *New* files do not: they are only linked at build time, so a newly added launch file, RViz config or map is simply absent and the tool that wanted it falls back to a default instead of reporting the miss. A brand-new `navigation.rviz` was silently replaced by RViz's factory layout exactly this way.

10.4 Localise the robot

AMCL starts with `set_initial_pose: false`, so until it is told where the robot is it publishes no `map→odom` transform and every goal fails to plan. Two ways to fix that:

1. **By hand, in RViz.** Click **2D Pose Estimate**, then click where the robot is and drag in the direction it faces.
2. **From ground truth.**

```
scripts/seed.sh
```

This reads the true pose from Gazebo, publishes it on `/initialpose`, and then rotates the robot in place.

Why `seed.sh` rotates afterwards, and why that is not cheating

A particle filter only updates when the robot has moved past `update_min_d` (0.25 m) or `update_min_a` (0.2 rad). A stationary robot therefore keeps whatever spread it was seeded with forever — the initial cloud is never corrected by a scan, because no update ever fires. Rotating in place is the cheapest way to force updates and cannot collide with anything. This is a legitimate initialisation, not a thumb on the scale, and it is reported: the validation script counts every re-seed and prints the total. In the corrected run that total is 0/61.

10.5 Watch what is happening

Terminal 5

```
scripts/watch.sh
```

prints, live: the true pose from Gazebo, the believed pose from `/amcl_pose`, the gap between them, and σ_x , σ_y , σ_θ . This is the same triple that equation (7.1) scores, shown continuously. Read the ground truth directly at any time with:

```
gz model -m romr -p
```

10.6 Run the evaluation

The suite

```
# quick smoke test - eight commands
scripts/validate_commands.py --mode graph --limit 8

# the full suite
OUT=results/validation_$(date +%F).csv
scripts/validate_commands.py --mode graph --out $OUT
```

Options:

Flag	Default	Meaning
-mode	graph	graph waits for the real navigation outcome and retries once with costmaps cleared; chat only dispatches
-out	validation_results.csv	output path
-limit	0 (all)	run only the first N trials
-timeout	300s	HTTP timeout per command
-seed	7	shuffle seed, so trial order is reproducible
-settle	3s	pause between trials
-arrival-tol	0.5m	the threshold of Part 7 §7.3

Requirements: the simulator, Nav2 and the backend must be up, and `gz` must be on `PATH` — without it there is no ground truth and the `error_true_m` column will be empty, which silently reduces the study to the very self-assessment it was designed to avoid. **Check that column is populated before trusting a run.**

10.7 Produce the figures

```
scripts/plot_validation.py results/validation_2026-08-04_fixed.csv \
                        results/figures/error_distribution.png
scripts/plot_results.py
scripts/plot_trace.py
scripts/particle_figure.py
```

10.8 Adding or moving a destination

Drive the robot to the spot, point it the way it should end up facing, then:

```
scripts/record_place.py garage
scripts/record_place.py parlour --alias "living room" --alias lounge
scripts/record_place.py --list
scripts/record_place.py --check      # audit every place against the map
```

The pose is taken from TF `map→base_link`, because that is the frame Nav2 plans in and therefore the only frame in which a goal means anything. Gazebo's ground truth is read as well, purely to report the gap: if localisation is off when you record a place, you bake that error permanently into the registry, and every future trial to that destination inherits it.

What was tuned, and what is left

11.1 The four faults that were fixed

Each fault is classified by which side of the importance-sampling identity (3.3) it sat on — a distinction that determines how seriously to take it.

Fault	Half	Was → Is	Exposed by
Sensor range cutoff	weight	8 m → 16 m	The fitted scanner reaches 16 m in a 13.85 m building
Rotational noise coefficient	proposal	0.20, then 0.25 → 0.02	The sweep of Part 5 §5.3
Hit-vs-noise weighting	weight	0.5/0.5 → 0.8/0.2	Scan-map agreement, Part 5 §5.4
Navigation outcome identity	—	untagged → per-goal epoch	The duration audit of Part 8 §8.3

Why the classification matters

A **proposal** fault degrades the estimator while still targeting the right posterior — it makes the filter inefficient. A **weight** fault changes *what is being estimated* — the filter converges confidently on the answer to a different question.

Two of the three model faults here were weight faults. That is consistent with what was observed: not a noisy estimate, but a confident and wrong one. It also means the fix could not have come from adding particles, adding compute, or waiting longer.

11.2 Tuning levers not yet exhausted

Each entry below states the current value, a candidate, the mechanism by which it should help, how to measure whether it did, and the risk of doing it. Nothing here is a recommendation to change blindly — the lesson of Part 5 §5.3 is precisely that plausible reasoning about these parameters can point the wrong way, and only a sweep settles it.

11.2.1 Lever 1 — the progress checker (highest expected value)

Current	SimpleProgressChecker, required_movement_radius: 0.3, movement_time_allowance: 10.0
Candidates	(a) PoseProgressChecker, which counts rotation as progress; (b) keep SimpleProgressChecker but raise movement_time_allowance above the worst observed final-rotation time; (c) relax yaw_goal_tolerance from 0.25 rad to 0.5 rad
Mechanism	Part 6 §6.4: the checker measures translation only, so a robot rotating on the spot at the goal registers no progress and is aborted
Expected effect	Recall 0.833 $\rightarrow$ close to 1.000; precision should stay at 1.000 because these trials genuinely arrived
Measurement	Re-run the full suite; count rows with <code>outcome != succeeded</code> and $e_{\text{true}} \leq 0.5$. Target: zero
Risk	(a) and (b) both weaken the stack's ability to detect a genuinely stuck robot — a robot spinning uselessly against a wall would now be tolerated for longer. (c) weakens the arrival criterion itself, which trades a reporting defect for a quality one. (a) was trialled during this project and reverted ; if it is revisited, the <i>stuck</i> case must be tested explicitly, not just the arrival case

11.2.2 Lever 2 — resampling interval

Current	resample_interval: 1 — resample on every update
Candidate	2 or 3
Mechanism	Part 4: AMCL uses multinomial resampling, which has no survival guarantee and count variance growing with J (equation (4.2)). Resampling less often lets weights accumulate information across several scans before diversity is spent
Measurement	Repeat the step-response protocol of Part 8 §8.10.2 and watch σ_θ after a 90° turn. Also log the effective sample size $J_{\text{eff}} = 1 / \sum_j (w^{[j]})^2$; the standard rule is to resample only when $J_{\text{eff}} < J/2$
Risk	Weight degeneracy in the other direction: if resampling is too rare, one particle takes all the weight and the filter is effectively running on a single hypothesis

11.2.3 Lever 3 — update thresholds during rotation

Current	update_min_d: 0.25, update_min_a: 0.2 (11.5°)
Candidate	update_min_a: 0.1 (5.7°), leaving update_min_d alone
Mechanism	Every measurement in this project says rotation is where the error enters and translation is benign — Part 2 §2.3.1. Correcting twice as often <i>during</i> the motion that causes the damage attacks the problem where it happens, rather than after
Measurement	Step response after a 90° turn: e , $\Delta\theta$, σ_θ . Also record CPU load, since this doubles the filter rate during turns
Risk	More frequent updates mean more frequent resampling, which interacts with Lever 2. Change one at a time

11.2.4 Lever 4 — sharpening the likelihood

Current	<code>sigma_hit: 0.2, max_beams: 120, z_hit: 0.8</code>
Candidates	<code>sigma_hit: 0.1; max_beams: 180</code>
Mechanism	The measured mean beam-to-wall distance at the true pose is 0.047 m and the median 0.050 m — both well inside the current $\sigma_{\text{hit}} = 0.2$ m. A Gaussian four times wider than the actual residual is a blunt instrument: it scores a pose 15 cm wrong almost as highly as the correct one. Halving it sharpens the peak in equation (5.4)
Measurement	The rotation study of Part 8 §8.10.1: recompute mean agreement at $\Delta = 0$ against $45^\circ, 90^\circ, 135^\circ, 180^\circ$ and check the contrast <i>ratio</i> improves, not just the absolute value
Risk	Real. A too-narrow σ_{hit} makes the likelihood so peaked that no particle scores well, weights collapse toward zero, and the filter loses track after a single unmodelled disturbance. 0.047 m is the residual <i>when correctly localised</i> ; the filter also has to score poses that are slightly wrong, and it needs to be able to grade them. Sweep it; do not jump

11.2.5 Lever 5 — using the covariance as a gate

Current	$\sigma_x, \sigma_y, \sigma_\theta$ are logged by <code>belief()</code> in the validation script but not acted on
Candidate	Refuse to report success, or trigger a re-localisation, when $\sigma_{\text{pos}} = \sqrt{\sigma_x^2 + \sigma_y^2}$ exceeds a threshold
Mechanism	Ground truth exists here and does not exist on a real vehicle. A self-assessment that predicts true error is the only signal that could carry this decision in deployment
Measurement	This is a study, not a setting. Plot σ_{pos} against e_{true} across the suite and compute the correlation. If they are uncorrelated, the covariance is useless as a gate and <i>that is a publishable negative result</i>
Risk / caveat	The corrected run has almost no spread in e_{true} (everything between 0.117 and 0.483), so there is nothing for σ to correlate <i>with</i> . The study needs deliberately induced failures — restore <code>laser_max_range: 8.0</code> , or raise α_1 — to generate the range. Part 5 §5.7 already warns that Σ measures particle agreement, not correctness, so the honest prior is that this will correlate weakly

11.2.6 Lever 6 — inflation, currently uncommitted

Current	<code>cost_scaling_factor: 6.0, inflation_radius: 0.55</code> — present in the working tree but not committed
Mechanism	Equation (6.1). A larger radius or a smaller scaling factor pushes paths further from walls, at the cost of refusing narrow doorways
Measurement	Two numbers together, never one: the fraction of the global costmap at or above the blocked value (currently 25.3%), and the planner’s success rate over all eight destinations from all eight starts
Risk	This interacts directly with localisation. A wider inflation makes the “believed pose lands in an inflated cell, planner refuses every goal” cascade of Part 7 §7.5 trigger at a smaller drift. Tuning inflation up to compensate for localisation error is treating the symptom

11.2.7 Lever 7 — the resampler itself

Current	Multinomial, fixed inside <code>nav2_amcl</code> , not configurable
Candidate	Fork <code>nav2_amcl</code> and implement stochastic universal sampling, accepting a fixed J in place of KLD adaptation
Mechanism	Part 4 §4.3.2–4.3.3: bounded count variance and a hard survival guarantee for any particle with $w \geq 1/J$
Measurement	With J fixed at 2000 for both arms, compare σ_θ after a 90° turn, and the number of distinct ancestors surviving after 50 updates
Risk	High effort, and it discards KLD adaptation — the filter would carry 2000 particles even when converged. This is a legitimate piece of future work and should be described as such, not attempted late. It is also the only item in this list that changes code outside this repository

11.3 A protocol for tuning anything on this list

Five rules, learned the hard way in this project

1. **Change one parameter at a time.** Two of the four faults above were found only because they were isolated. The intermediate run (`results/validation_2026-08-04.csv`) exists precisely to show the localisation fix and the outcome-identity fix were independent.
2. **Re-seed before every trial.** Otherwise you measure accumulated history rather than the parameter. `scripts/seed.sh`.
3. **Sweep, do not reason.** $\alpha_1 = 0.25$ was chosen by a physically correct argument about caster scrub and is the worst point on the curve. The parameter's units — variance per unit motion squared — do not match the intuition the argument appeals to.
4. **Score against ground truth, never against the estimate.** `gz model -m romr -p`. Anything else measures the robot's opinion of itself.
5. **Report the negative results.** The recovery-alpha experiment, the $\alpha_1 = 0.25$ trial and the reverted progress checker are all more informative than the settings that happened to work, because they say *why* the working settings work.

Glossary

Term	Meaning
AMCL	Adaptive Monte Carlo Localisation. A particle filter that estimates the robot's pose in a known map, with the number of particles adapting to how uncertain it is.
Bayes' rule	$P(A B) = P(B A)P(A)/P(B)$. The formula for updating a belief when evidence arrives. Equation (1.4).
Belief	The full probability density over where the robot might be, not a single position. Equation (2.3).
Costmap	A grid over the environment where each cell holds how bad it is to be there. Published on a 0–100 scale.
Covariance	A matrix describing how spread out the particles are, and how their errors correlate. Equation (5.6).
Dead reckoning	Estimating position by adding up small movements. Never self-corrects. Equation (2.2).
Density	A function whose <i>area</i> over an interval gives a probability. Can exceed 1; the total area is always 1.
Drift	Here, d_{drift} : the gap between where the robot believes it is and where it actually is.
DWB	Dynamic Window Approach local planner. Simulates a fan of short trajectories each cycle and picks the best-scoring one.
Expected value	The average of a quantity under a density. An integral, equation (1.2).
False positive	The stack reported success and the robot had not arrived. The central quantity of this study.
Footprint	The robot's outline as a polygon, rather than a single radius. ROMR: 0.521 m by 0.608 m.
Frame	An origin and set of axes. This project uses <code>map</code> , <code>odom</code> and <code>base_link</code> .
Ground truth	The real answer, obtained independently of the system under test. Here: <code>gz model -m romr -p</code> .
Importance sampling	Sampling from a convenient density and correcting with weights. Equation (3.3).
Inflation	A soft cost halo painted around obstacles so paths avoid hugging walls. Equation (6.1).
Integral	An area under a curve; equivalently, a total made of infinitely many infinitely small pieces.
KLD sampling	The rule that decides how many particles AMCL needs, from how spread out they currently are. Equation (4.4).
Likelihood	How plausible the observed sensor reading is, supposing a particular pose. Equations (5.3)–(5.4).
Mean / median	Average / middle value. The results report both, because they differ when there are outliers.
Multinomial resampling	The roulette-wheel method: J independent draws. What <code>nav2_amcl</code> uses. Part 4 §4.2.

Term	Meaning
Nav2	The ROS 2 navigation stack: planner, controller, recoveries, behaviour tree.
Normalisation	Dividing by a total so probabilities sum (or integrate) to 1. Equation (1.1).
Odometry	Position estimated from wheel rotation. Smooth but drifting.
Particle	One complete hypothesis about the pose: an (x, y, θ) triple plus a weight.
Particle depletion	Loss of diversity through repeated resampling, until every particle descends from one ancestor.
Posterior / prior	Belief after / before the evidence.
Precision / recall	Of the claims made, how many were true / of the opportunities, how many were claimed. Part 8 §8.8.
Proposal	The density you actually sample from, π . A design choice.
Resampling	Replacing the weighted particle set with an equally weighted one by drawing with replacement.
Standard deviation (σ)	Square root of variance; typical distance from the mean, in the original units.
Stochastic universal sampling	The comb method: one random offset, J equally spaced pointers. Lower variance, $O(J)$, unused by AMCL. Part 4 §4.3.
Support condition	The requirement that the proposal can generate samples everywhere the truth might be. Equation (3.6).
TF	The ROS transform system. The seam through which AMCL's answer reaches Nav2, carrying no confidence with it.
Variance	Mean squared distance from the mean. Measures spread.
Weight	A particle's share of the belief; in importance sampling, $w = p/\pi$. Forced, not chosen.

Sources for the reproduced material

Two items in this document are not original to the project and must be cited wherever they are reused.

1. **Figure 6.1, the Navigation 2 architecture diagram**, is reproduced from the Navigation 2 documentation. Cite Macenski et al., *The Marathon 2: A Navigation System*, IROS 2020, and the Navigation 2 documentation at <https://docs.nav2.org>.
2. **The source excerpts in Part 4 §4.4** are quoted from `nav2_amcl/src/pf/pf.c` on the `humble` branch of <https://github.com/ros-navigation/navigation2>. The particle-filter implementation descends from the Player/Stage project and carries its original authorship; the KLD sampling rule of equation (4.4) is due to Fox, *KLD-Sampling: Adaptive Particle Filters*, NIPS 2001.

The standard reference for Parts 1–5 is Thrun, Burgard and Fox, *Probabilistic Robotics* (MIT Press, 2005): Chapter 2 for the Bayes filter, Chapter 4 for particle filters and resampling, Chapter 5 for the motion models and the α coefficients, Chapter 6 for the beam and likelihood-field measurement models, and Chapter 8 for Monte Carlo localisation.

All measurements, all tables, all CSV files and every other figure in this document were produced by this project, in simulation.